

Filosofía Lean → Enfoque para procesos empíricos

Es una filosofía o enfoque más viejo que agile, que nace en Japón ('40) con Toyota, y **busca maximizar el valor generado al cliente con el mínimo consumo de recursos**, argumentando que todo el esfuerzo que no cree valor, se considera desperdicio.

La filosofía "Lean", busca atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente final quiere, cuando lo quiere, dónde lo quiere, a un costo mínimo y precio justo. **Siendo el cliente quien determina si el servicio o producto que la empresa entrega tiene valor o no.**

Para esto utiliza el método Just in Time, el cual permite el intercambio del producto entre fases del proceso de desarrollo o a la entrega del producto final al cliente, **justo a tiempo**, esto es poco antes de que lo usen y en la cantidad justa y necesaria.

7 principios Lean



- 1) **Eliminar desperdicios:** Debemos tener un proceso que sea eficiente y que sume valor en todos los pasos del proceso, todo paso o parte que no contribuye a la generación de valor produce un desperdicio.

Desperdicio: cualquier cosa que no agregue valor percibido por el cliente. Incluye cosas como:

- Requisitos súper detallados que nadie lee.
- Funcionalidades que se desarrollan por si acaso y nunca se usan.
- Software a medio hacer que no se puede entregar.

Busca identificar y eliminar las fuentes de desperdicio para evitar retrabajo y que los artefactos se vuelvan obsoletos antes de terminarlos.

Hay 2 tipos de desperdicios:

1. **No se pueden evitar:** Se minimizan para llevarlos a la mínima expresión posible. Por ejemplo: actividades de control ,son necesarias pero generan desperdicio.
2. **Se pueden evitar**

Este principio se relaciona con los principios agile 3, 7 y 10:

- 3: entregar software funcionando frecuentemente
- 7: el software funcionando es la principal medida de progreso)
- 10: la simplicidad o maximizar el trabajo no realizado es esencial: Porque en el software el trabajo parcialmente hecho y las funcionalidades extra son desperdicio (en industrias tangibles sería el inventario).

Los 7 + 1 desperdicios de Lean:

1. Características extras [producir productos extras]: Tiene que ver con:
 - a. **Funciones extra:** No son gratis, añaden complejidad que dificultan el mantenimiento, introducir cambios, aumentan el riesgo de errores y quedan obsoletas antes de que alguien las use de verdad.
 - b. **Up Front Specification (especificar anticipadamente):** cuando se intenta satisfacer esos requerimientos en etapas posteriores de implementación, la información de especificación está obsoleta, incompleta o errónea, debido a que cuando se especificó el requerimiento había incertidumbre o falta de información.
Esto se relaciona con el principio lean 4 (diferir compromisos hasta el último momento responsable).
2. Trabajo a medias [acumulación de stock]: Tanto Lean como Kanban trabajan con un enfoque binario. No hay porcentaje (%) de avance en las tareas, o está hecha o no está hecha. Todo lo que quedó a medio camino es desperdicio como: US, casos de uso, bugs, etc. sin terminar, lo que genera que deba ser retomado posteriormente para su finalización.
Esto se relaciona con el desperdicio de Cambio de Tareas y el principio Just In Time.
3. Proceso extra [Proceso extra]: Esto se refiere a los pasos extras que se ejecutan en el proceso de desarrollo que no aportan valor al producto. El objetivo es identificarlos y eliminarlos, para no destinar esfuerzo a actividades que no aportan valor.
4. Búsqueda de información [Búsqueda de información]: Esto termina siendo un problema importante y difícil de manejar si no se implementa la disciplina SCM, porque no contamos con información suficiente y/o de fácil acceso para tener una trazabilidad sobre los ítems de configuración, lo que dificulta identificar el impacto de los cambios y ralentiza la resolución de errores.
5. Defectos[defectos]: Son errores que se trasladan de una etapa a otra etapa posterior en la cual se introdujo. Esto provoca retrabajo, debido a que Testing debe “devolver” la funcionalidad a desarrollo para la resolución de defectos o bugs.
6. Esperas [costos de logística]: Esto es el tiempo de espera que una persona tiene que pasar para poder realizar su trabajo. Ese tiempo de espera puede ser causado porque depende del trabajo de otra persona, porque requiere aprobación de un superior, etc. (por eso la importancia de equipos multidisciplinarios → para evitar tiempos de espera).
7. Cambios de tareas (Multitasking) [Movimiento]: Como el desarrollo de software es un trabajo intelectual, uno de los desperdicios más grandes al hacer este tipo de trabajo, es el cambio de tareas o multitasking. Esto genera una pérdida de tiempo en prepararnos para realizar cada tarea (tiempo de seteo).
Estos dos últimos desperdicios tienen que ver con el JIT (Just In Time).
8. Talento no utilizado: No utilizar la experiencia, conocimiento y creatividad del personal.

Herramienta para identificar dónde están estos desperdicios: Mapeo del flujo de valor (Value String Mapping):

- **En la manufactura:** De todo un proceso que dura 319 días, desde que arranca hasta que llega al consumidor, quizás sólo se añade valor al producto durante 3 horas. El resto es tiempo de espera, transporte, etc.
- **En el software:** Mapeas el flujo desde que nace la idea o necesidad hasta que llega al usuario, generando valor. Esto te permite saber dónde se atasca el trabajo, donde hay esperas, cuellos de botella, etc.
 - Los mayores retrasos no suelen estar en la codificación o las pruebas, sino después, en las fases de aprobación, despliegue, en la burocracia interna.

La clave es ganar visibilidad, para entender en donde se pierde el tiempo y el esfuerzo que no le sirve de nada al cliente.

2) Amplificar el aprendizaje: Tiene que ver con la transparencia y transformación de conocimiento implícito en explícito, lo cual fortalece al equipo de trabajo. Crear y mantener una cultura que contribuya al aprendizaje colectivo. Todo conocimiento que se genere debe compartirse para que sea accedido por toda la organización.

Es un gran cambio respecto a la idea tradicional de planificar todo al detalle y luego ejecutar. Se distinguen dos conceptos:

- La producción: seguir un paso a paso para obtener el mismo producto una y otra vez.
- El desarrollo: crear el paso a paso por primera vez.

El desarrollo de software, de productos nuevos o innovadores, o de problemas complejos (mal estructurados), es un proceso de descubrimiento, de prueba y error.

El desarrollo es aprendizaje puro.

La herramienta para el aprendizaje es la retroalimentación (feedback).

- El modelo clásico en cascada → El feedback llega tardísimo, si es que llega
- Los ciclos iterativo-incremental → El feedback llega antes

Acá el refactoring es clave: permite que el diseño evolucione según se aprenda y se vayan tomando las decisiones más tarde.

Está muy asociado al siguiente principio: Decidir lo más tarde posible, cuya idea es mantener las opciones abiertas **hasta que tengas más información**, lo que demuestra la necesidad de amplificar el aprendizaje para tener la información necesaria antes de tomar decisiones.

Abrazar la incertidumbre reduce más el riesgo que intentar eliminarlo con predicciones al principio, que suelen ser erróneas. No es no planificar, sino planificar de forma adaptativa.

Se relaciona con los principios ágiles de (2) recibir requerimientos aún en etapas finales y el principio de (4) personas técnicas y no técnicas trabajando juntos todo el proyecto.

3) **Tomar decisiones tardías/ Diferir compromisos hasta el último momento responsable:**

Esto apunta a postergar la toma de decisiones lo suficientemente como para tener la mayor información posible para tomar dicha decisión, pero hasta antes de que ese momento sea muy tarde, ya que en ese caso otro tomará la decisión por mi o ya no valdrá la pena siquiera decidir.

Antes de tomar la decisión se van contrastando distintas opciones en paralelo. Apunta a tomar las decisiones lo más informado posible y en etapas tempranas sólo definir lo indispensable.

Esto se puede reflejar en Agile con el Product Backlog, donde nunca se tiene el 100% de los requerimientos, sino que se va completando a medida que se tiene más información sobre lo que se necesita implementar.

Pensamiento de opciones (Options Thinking): Piensan en las decisiones como si fueran operaciones financieras. Una opción te da el derecho pero no la obligación de hacer algo en un futuro.

El propio proceso secuencial genera una curva exponencial en el costo del cambio.
Cuanto más tarde se cambia más caro sale.

Pero si tu proceso no es secuencial, sino que es flexible y adaptativo, tu curva de costos es más plana.

**Retrasar las decisiones irrevocables que te cierran puertas, hasta el último momento responsable, cuando tenés la máxima información y el menor riesgo.
Confiar en la experiencia del equipo es más efectivo que intentar predecir y controlar todo al detalle.**

Se relaciona con los principios ágiles de maximizar el trabajo no hecho y de recibir cambios de requerimientos aun en etapas finales.

4) **Entregar lo antes posible:** La idea es entregarle al cliente software funcionando lo más rápido posible y que este producto sea útil para él. Esto trae ventajas:

- Para el cliente: recibe valor antes, puede dar feedback antes, tiene más flexibilidad para cambiar de opinión y recibe la funcionalidad antes de que cambie su contexto.
- Para el negocio: reducir el tiempo que tiene capital inmovilizado en “trabajo en progreso”, disminuye el riesgo de que el mercado cambie antes de que saques el producto, y acelera el ciclo de aprendizaje.

Es peligroso tener a todos ocupados el 100%

- **Tiempo de ciclo:** Tiempo total que demora una unidad de trabajo en pasar por todo el sistema, desde que empieza hasta que termina.
- Cuando el uso de un recurso se acerca al 100% el tiempo de espera se dispara exponencialmente ya que una pequeña sobrecarga causa demoras enormes. Se crean colas y cuellos de botella por todos lados.
- Para que el flujo sea rápido y flexible tenes que tener cierta **holgura en el sistema, capacidad no utilizada que pueda absorber la variabilidad normal del trabajo.**

Para lograr esta rapidez sostenible se usa el sistema PULL:

PUSH	PULL
El trabajo se planifica de manera central y se empuja a la próxima estación cuando toque. No importa si la estación está lista o sobrecargada.	Una estación solo jala (hace PULL) de trabajo cuando tiene capacidad disponible.
	¿Cómo se ve en el desarrollo de software?: • Ves las tarjetas del tablero de Kanban fluir de uno al otro • Se ponen restricciones al WIP (Work in progress) para evitar cuellos de botella.

Si entregamos rápido ¿qué entregamos primero? El que tenga más costo de retraso. El costo de retraso es el número monetario de cuánto dinero pierde el negocio porque una funcionalidad no esté disponible para el cliente.

La meta no es mantener a la gente ocupada, sino acelerar el flujo de valor real. Y cuantificar el costo del retraso para tomar decisiones inteligentes de que entregar primero.

Para esto Lean propone:

- Acotar ciclos de desarrollo (principio 1 de Agile: satisfacer al cliente con entregas tempranas)
 - Entregar rápidamente incrementos pequeños de valor (principio ágil de entregas tempranas y frecuentes de software de valor para el cliente).
- 5) Potenciar al equipo:** El equipo debe ser multifuncional y autogestionado, y sus miembros deben estar capacitados y motivados.
- Esto implica por parte de los superiores: no subordinar (esto anula las capacidades intelectuales), entrenar líderes, delegar decisiones y fomentar buena ética laboral.
 - E implica por parte de los miembros del equipo: compromiso para desempeñarse y cumplir con lo pactado de forma completa y correcta.

Este aspecto termina siendo un talón de Aquiles en las filosofías Agile y Lean, ya que en la práctica pocas veces se logran reunir todas estas características.

Advierten en que si los programas de iniciativas de mejoras de procesos como el CMM (Capability Maturative Module) o la certificación ISO 9000 se convierten en cargas burocráticas que cierran a los desarrolladores a seguir el proceso documentado, en lugar de habilitar a las personas a que piensen y mejoren, pueden ser contraproducentes.

¿Cómo se motivan los equipos?

- Contrata buenas personas y déjalas en paz.
- 15% de tiempo para proyectos personales

- Fomentar que haya campeones que empiezan desde abajo
- Gente apasionada que impulsa ideas
- Resaltar motivadores que van más allá del sueldo
- Tener un propósito con tu trabajo, sentir que aportas valor y que tu trabajo importa.
- Pertenencia a un equipo
- Seguridad psicológica: probar cosas nuevas y cometer errores sin miedo al castigo
- Desarrollar la competencia, y sentir que mejoras en lo que haces
- Trabajar con altos estándares técnicos, ver cómo mejora la calidad
- Tener un feedback claro.

Rol de gestor	Rol de líder	Master developer
• Manejan la complejidad • Organizan • Planifican	• Inspiran • Mantienen el cambio • Manejan la visión	No es un jefe o manager, es un líder técnico muy respetado. Conoce mucho sobre lo técnico y sobre el dominio negocio. • Guía al equipo • Facilita que el diseño emerja • Actúa como mentor

**Foco en las personas cómo interactúan no solo en los procesos y las herramientas.
El respeto, la autonomía, la confianza y la inversión de que la gente desarrolle maestría son cosas mucho más efectivas para lograr resultados excepcionales que el control jerárquico y la estandarización forzada.**

Relacionado con el principio 11 del Manifiesto Ágil (“Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados”).

- 6) Crear la integridad:** La calidad del producto no se negocia y la simplicidad es un factor clave. El producto tiene que satisfacer las expectativas y necesidades del cliente, esa es la principal prioridad y **cada decisión se alinea con esto, no se debe perder de vista que esa es la prioridad principal.**

Para esto se deben encastrar todas las partes del producto que tengan coherencia y consistencia para hacer al producto más íntegro; donde los requerimientos que cubren la solución se observan como un todo cohesionados armónicamente.

Tenemos dos tipos de integridad:

- Integridad percibida: La experiencia del cliente, el producto es coherente y útil como si leyera mi mente. Lo que ve y siente el usuario
- Integridad conceptual: Referido a lo interno, la coherencia, armonía del sistema por dentro, las partes encajan bien, la arquitectura es sólida y fácil de entender.

Los componentes clave para lograr integridad es:

Tener un excelente flujo de información:

- **Integridad percibida:** Un flujo rico y bidireccional entre el cliente y el equipo. Para que el producto haga lo que el cliente necesita.
- **Integridad conceptual:** Un flujo de información igual de bueno entre los desarrolladores y los diferentes componentes del sistema. Acá se critica mucho lo secuencial porque cada vez que se pasa de un wf al otro cambian los especialistas que tenían la info importante y conocen los matices.

¿Cómo mejorar el flujo de información?

- Equipos pequeños y multifuncionales que duren en el tiempo
- Con acceso directo y frecuente al cliente a sus representantes
- Pruebas de aceptación escritas por el cliente o junto con el cliente
- Modelos compartidos visuales que todos en el equipo entiendan
- El master developer como guardián de la visión global del sistema.

Refactoring constante:

Refactoring: Mejora continua de la estructura interna del código sin cambiar su funcionamiento. No busca corregir bugs sino pagar la deuda técnica (ese desorden que se acumula con el tiempo y hace que cada vez sea más difícil y arriesgado hacer cambios). Es una inversión para mantener al código fácil de entender, sano y flexible.

El refactoring busca mejorar las siguientes características de un buen diseño interno:

- **Simplicidad:** evitar la complejidad innecesaria.
- **Claridad:** que sea fácil de entender para otro desarrollador.
- **Adecuación al uso:** que resuelva bien el problema que tiene que resolver.
- **Sin repetición:** DRY
- **Que no tenga funciones innecesarias:** YAGNI

El refactoring constante ayuda a mantener estas cualidades a lo largo del tiempo.

Testing: No solo encuentra errores.

<u>Pruebas que hace el desarrollador:</u>	<u>Pruebas que se hacen a nivel de cliente:</u>
● Unitarias ● De integración	● Funcionales ● De aceptación

Las pruebas cumplen múltiples roles esenciales:

- Son una forma de comunicación.
- Dan retroalimentación rápida sobre si el código funciona como se espera.
- Son una red de seguridad para hacer cambios con confianza sin miedo a romper algo porque las pruebas te avisan.
- Sirven como documentación viva, describen cómo funciona el sistema ahora, no como se lo planteó hace meses.
- Facilitan el mantenimiento a largo plazo.

La integridad no se agrega al final, se teje en el proceso a través de comunicación fluida, código limpio con refactoring y pruebas continuas que te den confianza.

- 7) **Visualizar todo el conjunto:** En Lean se busca tener una visión holística, que permita asociar y comprender el todo, el producto y el valor agregado que hay detrás o el servicio que brinda el complemento de los productos. Todo más allá de los objetivos particulares por áreas o gerencias, porque **la idea es que los objetivos particulares de las partes estén alineados con los objetivos globales.**

Acá entra el pensamiento sistémico: No puedes entender o mejorar un sistema complejo mirando solo las partes por separado. **El todo es más que la suma de las partes.** Las partes interactúan, hay dependencias, hay bucles de retroalimentación, etc.

Patrones sistémicos comunes a tener en cuenta:

- **Límites al crecimiento:** un factor concreto limita el crecimiento de todo el sistema.
- **Desplazamiento de carga:** aplicas una solución rápida y fácil a corto plazo que enmascara el problema y lo perjudica a largo plazo.
- **Sub optimización:** optimizar las partes sin pensar en el resultado del todo perjudica el rendimiento general del sistema.

La suboptimización es muy peligrosa en las métricas. Las métricas locales pueden perjudicar al sistema completo.

Medir el desempeño individual en trabajos en la industria del conocimiento (que suelen ser en equipo) suele ser contraproducente. La gente va a hacer lo necesario para que la métrica salga bien, incluso si eso perjudica el objetivo global.

“Cuando una métrica se transforma en un objetivo deja de ser una métrica.”

Factores sistémicos que impactan el comportamiento de las personas:

- Demasiada presión
 - Falta de herramientas
 - Código heredado imposible
 - Contratos:
 - Los contratos tradicionales con **precio fijo** protegen a los clientes del sobre costo pero condicionan al proveedor a estar revisando el alcance de manera muy rígida, a decir no a cualquier cambio y potencialmente a bajar la calidad para cumplir el presupuesto si o si.
 - El **tiempo y material fijo** protege al proveedor del riesgo del alcance pero le pasa todo el riesgo del costo al cliente, y este se ve obligado a microgestionar las horas, el gasto, etc.
- ¿Qué se recomienda? Contratos colaborativos:
- **Contrato multietapas:** con puntos decisión y renegociación a medida que se avanza y se aprende.
 - **Modelos de costo objetivos o plazos objetivos:** ambas partes comparten el riesgo y recompensa de si se superan o no los objetivos.

La clave es no fijar un alcance rígido al inicio, ya que este va a cambiar. Y en lugar de eso enfocarse en entregar valor incremental alineando los intereses de ambas partes hacia el éxito del producto final.

Método Kanban en el contexto del desarrollo de software

Kanban es un método para gestionar todo tipo de servicios profesionales, también el denominado trabajo del conocimiento. Utilizar el Método Kanban significa enfocarse en mejorar los servicios desde la perspectiva de sus clientes. **Es un enfoque para la gestión de formas de trabajo (procesos) que busca obtener mejora continua.**

¿Kanban es una metodología o marco de trabajo?

- **Metodología:** Contienen flujos de trabajo y procesos definidos con sus roles y responsabilidades. Específicos a un dominio, como el del desarrollo de software.
- **Marco de trabajo:** Es una metodología incompleta. Un conjunto de estructuras destinadas a tener una aplicación más amplia, pero que requiere personalización con el fin de adaptarse a cada contexto.

Kanban no es una metodología ni un marco de trabajo. Es un método de gestión que debe aplicarse al uso de la metodología, del marco de trabajo o de la manera de trabajar actual.

Kanban es un medio para mejorar el qué y el cómo se realizan las cosas. Este, no sustituye a lo que ya se hace

Orígen

Este enfoque surge con estudios de Toyota, para mejorar las técnicas de almacenamiento y tiempo de stockeo en supermercados. Toyota hacía que el proveedor revisara e hiciera el control de calidad de la mercadería en su planta, y que la hiciera llegar a la línea de producción exactamente cuando la necesitaba para producir.

Es el **Just in Time original**. En el software se trata de especificar requerimientos solo cuando estos vayan a ser necesarios.

Valores de Kanban:

- **Transparencia:** La creencia de que compartir información abiertamente mejora el flujo de valor de negocio. Utilizar un lenguaje claro y directo es parte del valor.
- **Equilibrio:** El entendimiento de que los diferentes aspectos, puntos de vista y capacidades deben ser equilibrados para conseguir efectividad. Algunos aspectos (como demanda y capacidad) causarán colapso si no se encuentran equilibrados por un periodo prolongado.
- **Colaboración:** Trabajar juntos. El Método Kanban fue formulado para mejorar la manera en que las personas trabajan juntas, por ello, la colaboración está en su corazón.
- **Foco en el cliente:** Conociendo el objetivo para el sistema. Cada sistema kanban fluye a un punto de valor realizable — cuando los clientes reciben un elemento solicitado o servicio. Los clientes en este contexto son externos al servicio, pero pueden ser internos o externos a la organización como un todo. Los clientes y el valor que estos reciben es el foco natural en Kanban.

- **Flujo:** La realización de ese trabajo es el flujo de valor, tanto si es continuo como puntual. Ver el flujo es un punto de partida esencial en el uso de Kanban.
- **Liderazgo:** La habilidad de inspirar a otros a la acción a través del ejemplo, de las palabras y la reflexión. Muchas organizaciones tienen diferentes grados de jerarquía estructural, pero en Kanban, el liderazgo es necesario a todos los niveles para alcanzar la entrega de valor y la mejora.
- **Entendimiento:** Principalmente conocimiento de sí mismo (tanto individual como de la organización) para ir hacia adelante. Kanban es un método de mejora, por lo que conocer el punto de inicio es la base de todo.
- **Acuerdo:** El compromiso de avanzar juntos hacia los objetivos, respetando — y donde sea posible, acomodando — las diferencias de opinión o aproximaciones. Esto no es gestión por consenso sino un compromiso dinámico para mejorar.
- **Respeto:** Valorando, entendiendo y mostrando consideración por las personas. De manera apropiada al pie de esta lista se encuentra la base sobre la cual reposan el resto de los valores.

Principios de Kanban:

Gestión del cambio	Principios de Entrega del Servicio
● Comenzar con lo que haces ahora. ● Acordar la búsqueda de la mejora a través del cambio evolutivo. ● Fomentar actos de liderazgo a todos los niveles Kanban no es una transformación revolucionaria, utiliza un enfoque de cambio evolutivo, basándose en lo que ya tienes: → Entender los procesos actuales tal y como están siendo realizados en la actualidad → Respetar los roles actuales, las responsabilidades de cada persona y los puestos de trabajo → Las herramientas ya usadas → La forma de trabajo ya existente buscan mejorarla a través de la retroalimentación y la colaboración introduciendo mejoras graduales a un proceso de desarrollo de software o a una metodología de administración de proyectos <u>ya existente en una organización.</u> Esto permite reducir la resistencia al cambio por parte de las personas. El Método Kanban genera un <u>cambio evolutivo en</u> a través de los <u>conocimientos adquiridos</u> por las personas que trabajan con el Tablero Kanban y realizando <u>actos de liderazgo</u> para mejorar continuamente su forma de trabajar.	Los Principios de Orientación al Servicio son: ● Comprender y enfocarse en cumplir las necesidades y expectativas del cliente ● Gestionar el trabajo; dejar que los trabajadores se auto organicen en torno a él. ● Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados entregados

Prácticas generales de Kanban

- 1) **Visualizar:** Es clave para tener una colaboración eficaz e identificar oportunidades de mejora. Visualizar el trabajo y cómo fluye mejora mucho la transparencia. La visualización habilita la cooperación, ya que todo el mundo tiene, literalmente, la misma imagen. *Más detalles sobre la visualización se presentan en la sección de Tableros Kanban*
- 2) **Limitar el trabajo en curso (WIP):** El Trabajo en Curso nos indica el número de elementos de trabajo en un determinado momento. Los sistemas eficaces son los que se centran más en el flujo de trabajo y menos en tener ocupados a los trabajadores. Además, tener demasiado trabajo no finalizado es un desperdicio de tiempo, de dinero y alarga los tiempos de entrega.

Cuando los recursos están ocupados completamente, **no hay holgura en el sistema** y como resultado el flujo es deficiente, ya que una mínima sobrecarga genera cuellos de botella.

Para esto usamos el esquema de asignación de trabajo PULL o **sistema de arrastre**. *Esta práctica está relacionada con el principio Lean de **Entregar lo antes posible**. Donde se habla de entregar rápido buscando la eficacia y no explotando la capacidad de rendimiento al 100%*

- 3) **Gestionar el flujo:** El objetivo de gestionar el flujo de trabajo es poder terminar el trabajo de la forma más fluida y predecible posible, mientras se mantiene un ritmo sostenido. El seguimiento y la medición del flujo de trabajo nos da información para gestionar las expectativas de los clientes, hacer predicciones y mejorar.
- 4) **Hacer explícitas las políticas:** Las políticas organizan el trabajo, cada equipo tiene políticas diferentes. Los acuerdos del mismo equipo son una buena forma de introducir las políticas.

Todas estas políticas **deben ser acordadas** entre todas las partes involucradas, incluyendo a los clientes, interesados y trabajadores responsables del trabajo que está en el tablero. Las políticas **deben estar expuestas en un lugar destacado**, preferiblemente cerca del tablero.

Estas incluyen:

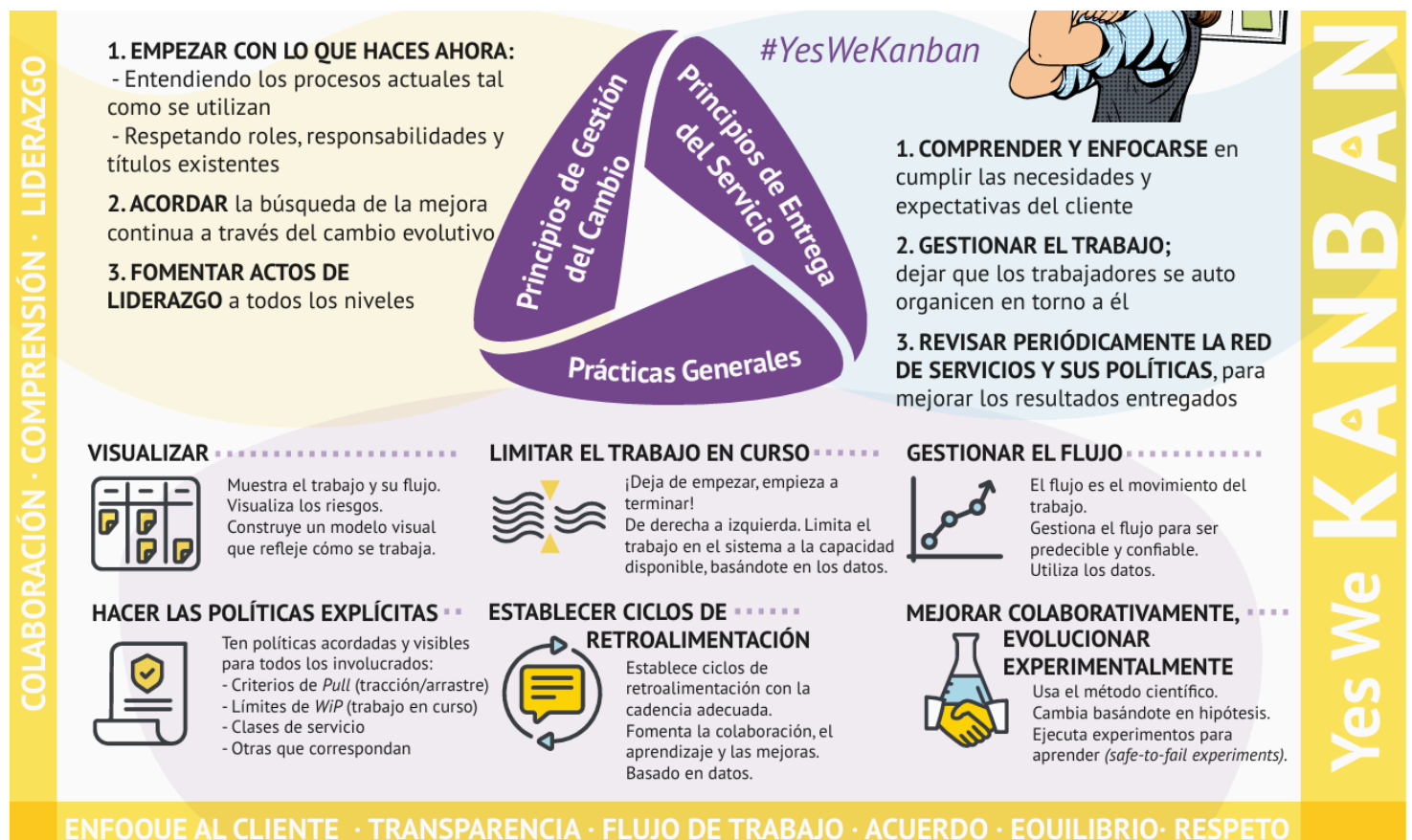
- Políticas sobre la reposición de trabajo en el tablero (cuándo, cuánta cantidad, por quién...)
- Definición de cuándo una actividad está terminada y ese elemento de trabajo puede continuar a través del flujo (criterios de tracción)
- Límites del trabajo en curso (WiP limits)
- Políticas para gestionar elementos de trabajo de diferentes clases de servicio
- Horarios y agenda de las reuniones
- Otros principios y acuerdos de colaboración

Las políticas deben ser: • pocas • sencillas • bien definidas • visibles • aplicables en todo momento • fácilmente modificables por los que prestan el servicio

- 5) **Implementar ciclos de retroalimentación:** Los ciclos de retroalimentación son necesarios para una entrega coordinada con el cliente y para mejorar la entrega de tu servicio. Un conjunto de ciclos de retroalimentación refuerzan el aprendizaje de la organización y su evolución mediante experimentos gestionados.

Recursos habitualmente utilizados en los ciclos de retroalimentación de los sistemas Kanban: tableros, métricas, y cadencias (conjunto de reuniones y revisiones periódicas).

- 6) **Mejorar colaborativamente, evolucionar experimentalmente:** Volviendo a los Principios de Gestión del Cambio, en el Método Kanban “comenzamos con lo que hacemos ahora” y “acordamos la búsqueda de la mejora a través del cambio evolutivo”. Aquí es donde la retroalimentación y las métricas son importantes para guiarnos en el camino de la evolución. Usamos el método científico para diseñar experimentos en entornos donde fallar es seguro, de forma que si nuestra hipótesis es correcta mantenemos el cambio; pero si el resultado no es positivo, podemos fácilmente volver al estado anterior.



En definitiva Kanban aprovecha los principios de Lean:

- Definiendo el valor desde la perspectiva del cliente.
- Limitando el trabajo en proceso WIP.
- Identificando y eliminando desperdicios.
- Identificando y eliminando las barreras en el flujo, es decir, todo lo que atrasaría el proceso: Relacionado con el principio de lograr entregar lo antes posible.
- Cultura de mejora continua.

¿Cómo aplicar kanban? - Armando el tablero

- Los **tableros Kanban** son el medio más habitual para visualizar un sistema Kanban.
- En un sistema Kanban, hay al menos un punto de compromiso y de entrega claro, así como una representación de la cantidad de trabajo permitido (Trabajo en curso o WiP).



- Los **elementos de trabajo** pueden ser de diferentes tipos y tamaños y habitualmente los encontramos como anotaciones individuales, conocidos como tarjetas o tickets.



- La serie de actividades por la que pasan los elementos de trabajo se denomina **flujo de trabajo**.
- Kanban se basa en el principio de “Comienza con lo que haces ahora”, por lo que el **tablero Kanban modela el flujo real (no una visión futura deseada)**.
- Los pasos individuales en el flujo de trabajo y los buffers son mostrados en **columnas**. Los **carriles** son usados por diferentes tipos de trabajo, proyectos, etc. para distribuir la capacidad.
- **El límite del WiP**, es decir, el número máximo de elementos de trabajo permitidos a la vez, se puede definir por estado(s) de trabajo, por persona, por carril, por tipo de trabajo, para todo el sistema Kanban, etc.
- **Limitar la entrada de trabajo en el sistema** es clave para reducir las demoras y cambios de contexto que pueden generar poca puntualidad, baja calidad y desperdicio. Limitar lo que entra a esta columna:
- Una señal de tracción virtual se genera cuando un límite de WiP no está completamente cubierto. **Mientras el trabajo se mueve hacia la derecha, la señales de tracción se mueven hacia la izquierda.**

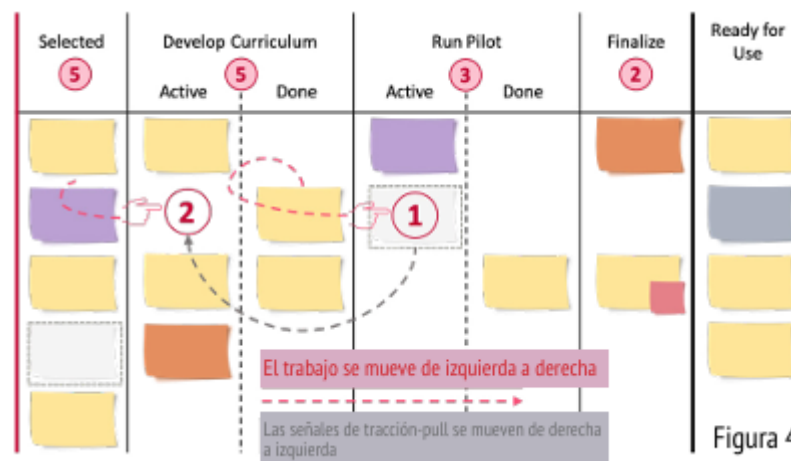
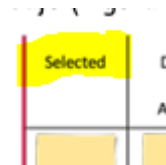


Figura 4

¿Cómo armamos un sistema Kanban?

1. **Entender el proceso de desarrollo actual** que se está utilizando.
2. **Modelar el proceso:** Definir los pasos y asignarle una columna a cada uno.

Tenemos columnas de:

- a. **En progreso**
- b. **Columnas de acumulación:** estas permiten el sistema de arrastre.

Cola de Producto	Análisis		Desarrollo		Listo para Build	En Testing		En Producción
	En progreso	Hecho	En progreso	Hecho		En Progreso	Listo para Despliegue	

3. Dividir el trabajo en piezas
4. Identificar las **unidades de trabajo** a utilizar (US, CU, deuda técnica, defectos, etc) asignando la capacidad en función de la demanda.

Cola de Producto
Casos de Uso 60 %
Mantenimiento 30 %
Defectos 10%

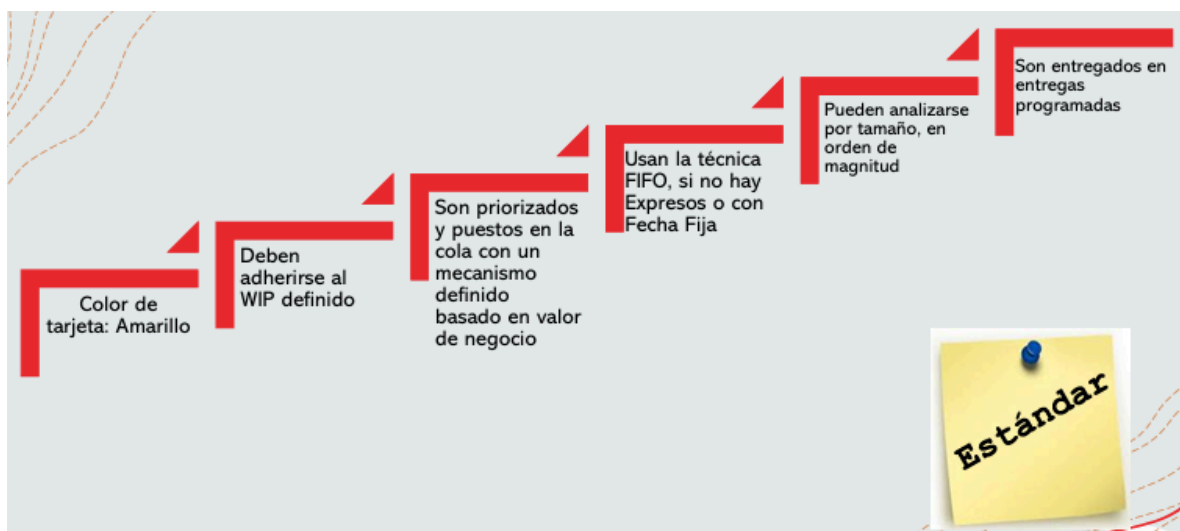
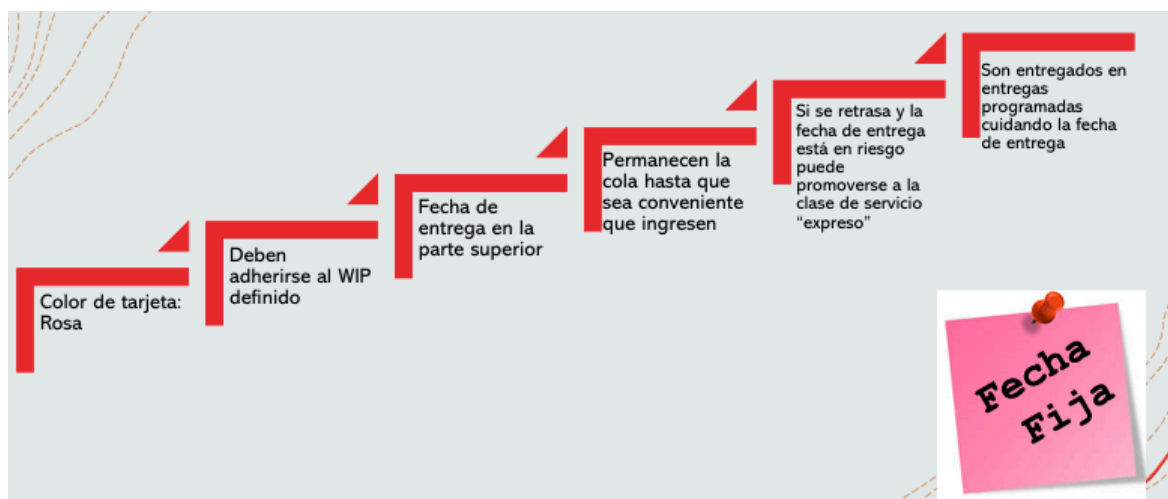
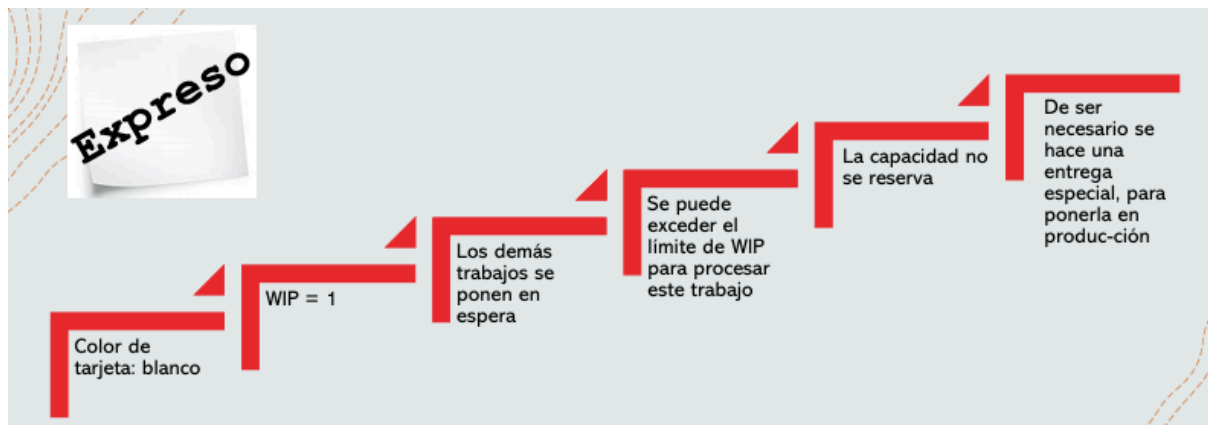
5. Definir el WIP:

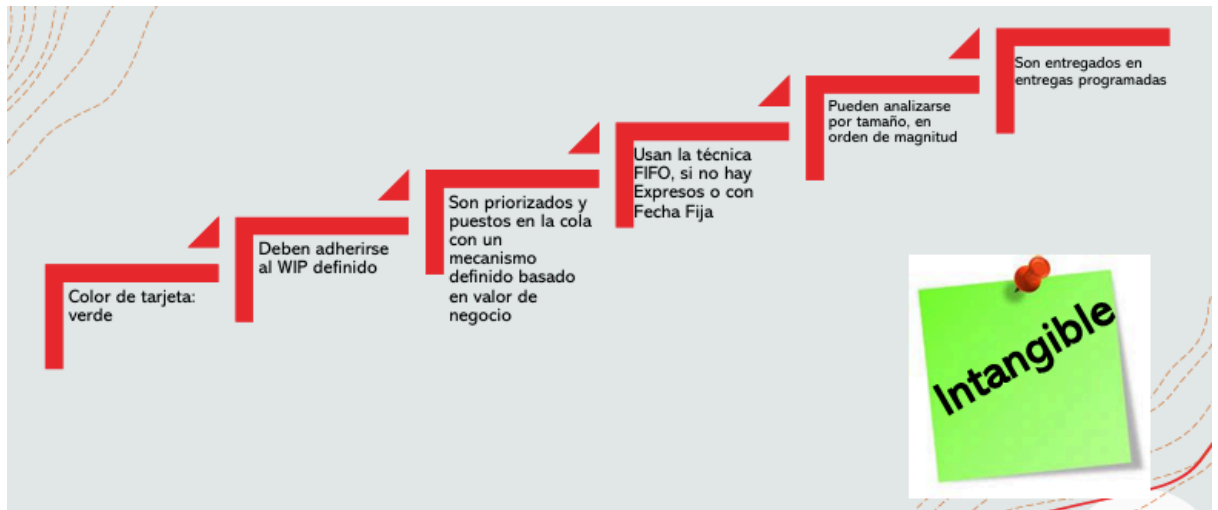
5	4	4	4	2	Total = 19			
Cola de Producto	Análisis		Desarrollo		Listo para Build	En Testing		En Producción
	En progreso	Hecho	En progreso	Hecho		En Progreso	Listo para Despliegue	

6. **Identificar clases de servicio:** Las más comunes son: Expreso, Fecha Fija, Estándar e Intangible.
7. **Visualizar el flujo de trabajo** diseñando un tablero representando el flujo de trabajo a través de columnas.

8. **Definir políticas y cadencias para cada clase de servicio identificada:** Esto incluye todo lo que las políticas deben abarcar (lo expliqué antes):

- Reposición de trabajo en el tablero (cuándo, cuánta cantidad, por quién...)
- Definición de cuándo una actividad está terminada y ese elemento de trabajo puede continuar a través del flujo (criterios de tracción)
- Definir políticas de calidad sobre: Límites del trabajo en curso (WiP limits)
- Cadencia de las reuniones (Horarios y agendas)
- Otros principios y acuerdos de colaboración como colores o indicadores visuales para las distintas clases de servicio
- Políticas para gestionar elementos de trabajo de diferentes clases de servicio





9. **Identificar cuellos de botella y resolverlos** (asignando más recursos o ajustando el WiP donde corresponda).

Ejemplos:

Tipo de política	Ejemplo
Entrada al sistema	“No se agrega nuevo trabajo si alguna columna está en su límite de WIP.”
Definición de “Hecho” (DoD)	“Una tarjeta se mueve a <i>Done</i> solo si pasó los tests automatizados y revisión por pares.”
Bloqueos	“Una tarjeta bloqueada se marca en rojo y debe discutirse en la próxima reunión de flujo.”
Clases de servicio	“Las tareas Expedite pueden romper el WIP, pero solo una a la vez.”
Cadencias	“Revisamos métricas de flujo todos los jueves y ajustamos WIP si el lead time supera 10 días.”

Diferencia	SCRUM	KANBAN
Objetivo	Eficiencia del sistema completo	Satisfacción del cliente mediante iteraciones
Enfoque principal	Eliminar desperdicios y optimizar procesos	Entregar valor rápidamente y adaptarse al cambio
Tiempo	Las iteraciones son de tiempo fijo.	El tiempo fijo es opcional. La cadencia puede variar. Pueden estar marcadas por la previsión de los eventos en lugar de tener un tiempo prefijado.
Compromiso	El equipo asume un compromiso de trabajo por iteración.	El compromiso es opcional.
Métrica por defecto	Para la planificación y mejora es la velocidad .	El por defecto es Lead Time (tiempo de entrega promedio)
Equipos	Multifuncionales	Multifuncionales o especializados.
Diagramas de seguimiento	Deben emplearse gráficos Burndown	No se prescriben diagramas de seguimiento.
Limitación WIP	Es indirecta y está marcada por el sprint. Timebox.	Es directa para cada etapa de trabajo o el estado del trabajo.
Agregar trabajo.	No se puede agregar alcance una vez comenzado el sprint.	Siempre que haya capacidad disponible, se puede agregar trabajo.
Estimaciones.	Se deben realizar estimaciones.	La estimación es opcional.
Compartido entre equipos	Cada equipo tiene su sprint backlog.	Todos los equipos comparten la misma pizarra o tabla kanban.
Permanencia tablero	En cada sprint se limpia.	El tablero es persistente.
Roles	Se prescriben tres roles. PO, SM y Equipo.	No se prescriben roles.
Priorización	El product backlog debe estar priorizado.	La priorización es opcional.

Funcionalidades	Espera que las funcionalidades se dividan tal que se completen en un sprint.	No se prescribe el tamaño de la funcionalidad.
-----------------	--	--

Framework para escalar Scrum/ Nexus

Dado que Scrum impone un límite en la cantidad máxima de integrantes del Equipo de Desarrollo, surge la necesidad de **escalar este Framework para poder gestionar productos de mayor envergadura**.

Existen diferentes frameworks para escalar como Nexus, scale scrum, less, SAFe.

Framework Nexus

Nexus es un Framework que consiste en roles, eventos, artefactos y técnicas que vinculan el trabajo de aproximadamente tres a nueve equipos Scrum, **trabaja sobre un único Product Backlog común para la construcción de un incremento** integrado de producto "Terminado". Con un solo Product Owner para entregar un único producto.

Surge para lidiar con la complejidad que supone varios Equipos Scrum trabajando sobre un mismo Product Backlog. Esta complejidad está dada por las siguientes dependencias entre equipos:

- Requerimientos: el alcance de estos puede superponerse. La forma en que se implementan también puede afectar a los demás.
- Conocimiento del dominio: el conocimiento del sistema de negocio debería mapearse a los Equipos Scrum para minimizar las interrupciones entre los mismos durante el Sprint.

Nexus = Scrum + mecanismos de integración

→ Mantiene los roles, artefactos y eventos de Scrum. Pero agrega mecanismos de coordinación e integración para evitar conflictos entre equipos y mejorar la entrega conjunta.

Responsabilidades:

- **Nexus Integration Team**: Responsable de asegurar que se produzca un incremento integrado al menos una vez en cada sprint.
- **Product Owner**: Es responsable de maximizar el valor del producto y el trabajo realizado e integrado por los Scrum Teams en un Nexus. También es responsable de la gestión eficaz del Product Backlog.
- **Scrum Master**: responsable de asegurar que el marco de trabajo Nexus se entienda y se promulgue como se describe en la Guía de Nexus.
- **Uno o más miembros del Nexus Integration Team**: a menudo está formado por miembros de los Scrum Teams que ayudan a los Scrum Teams a adoptar herramientas y prácticas que contribuyen a mejorar su capacidad para entregar un Integrated Increment útil y de valor que cumple con la Definición de Terminado.
Se hace hincapié en que todos los equipos deben integrar sus entregas frecuentemente (idealmente, a diario), para detectar problemas a tiempo.

Eventos (Nexus extiende o adapta los eventos de Scrum para un entorno con múltiples equipos):

Orden	Evento Nexus	Es..	Que lo diferencia	¿Cuándo?
-	Nexus Sprint	Mismo que en Scrum	• Los scrums teams producen un solo Integrated Increment (Incremento de integración). • Inicia con una única planificación y termina con una única revisión, centrada en un Incremento Integrado. • Contiene todos los eventos Nexus. • Se desarrollan ítems del Product Backlog distribuidos entre los Scrum Teams. • Se mantiene el foco en el Objetivo del Nexus.	Dura de 2 – 4 semanas como el Sprint en Scrum
1	Refinamiento entre equipos	Evento Nuevo [opcional]	• No está en Scrum base. • Se hace continuamente para identificar y reducir dependencias entre equipos. • Ayuda a los Scrum Teams a prever qué equipo entregará qué elementos del Product Backlog (PB). Para ello descompone el PB para que las dependencias sean transparentes y los distribuye. • Resultado: Product Backlog más claro y alineado para todos los equipos, facilita la planificación.	Continuo / Previo al Sprint
2	Nexus Sprint Planning	Extensión	• Participan representantes de todos los equipos y el PO. • Se seleccionan los ítems del PB para todos los equipos. • Se define el Objetivo del Nexus (objetivo general que guía al trabajo conjunto de los equipos hacia un incremento integrado). • Luego se construye el Nexus Sprint Backlog que surge de la selección de ítems del Product Backlog orientado por el objetivo del Nexus, y es influenciado por el refinamiento previo entre equipos. • Por último, cada equipo luego define su Objetivo de Sprint y su Sprint Backlog individual.	Comienzo del Sprint
3	Nexus Daily Scrum	Extensión	Reunión diaria con representantes de cada equipo. Se enfoca en problemas de integración e inspecciona el progreso hacia el objetivo del Sprint del Nexus. Complementa la Daily Scrums individual de cada equipo	Final del Sprint
4	Nexus Sprint Review	Extensión	Evalúa el Incremento Integrado generado por todos los equipos durante el sprint, devuelve retroalimentación y determina adaptaciones futuras. Reemplaza las Sprint Reviews individuales de c/equipo. pq ahora tenemos objetivo de nexus no de sprint	Final del Sprint
5	Nexus Sprint Retrospective	Extensión	Planifica formas de mejorar la calidad y eficacia en todo el Nexus (más allá de lo individual), e inspecciona cómo	Final del Sprint

			fue el último Sprint. Es complementada con las retrospectivas individuales de cada equipo. Se centra en mejoras transversales al Nexus.	
--	--	--	---	--

Artefactos

El Nexus Integration Team trabaja con los Scrum Teams dentro de un Nexus para **garantizar que se logre la transparencia en todos los artefactos** y que el estado del Integrated Increment se entienda.

Cada artefacto contiene un compromiso, y estos existen para reforzar el empirismo y el valor de Scrum para el Nexus.

- **Product Backlog:** Hay uno solo que contiene la lista de todo lo que el Nexus y sus Scrum Teams necesitan para mejorar el producto. El Product Backlog debe entenderse con tal nivel que permita detectar y minimizar las dependencias. Es responsabilidad del Product Owner.

→ El compromiso es el Objetivo del Producto, que describe el estado futuro del producto y sirve como un objetivo a largo plazo para el Nexus.

- **Nexus Sprint Backlog:** Está compuesto por el Objetivo del Sprint de Nexus y elementos del Product Backlog de cada Scrum Team del Nexus.

Se usa para resaltar las dependencias y el flujo de trabajo durante el Sprint. Se crea y ajusta en la Nexus Sprint Planning, se actualiza cuando se adquiere nuevo conocimiento durante el sprint, y se inspecciona en la Nexus Daily Scrum.

→ Su compromiso es el Objetivo de Sprint de Nexus, que es el único objetivo para Nexus. Es la suma de todo el trabajo y los Objetivos de Sprint de los Scrum Teams dentro del Nexus.

- **Integrated Increment:** Es la suma actual de todo el trabajo integrado completado por un Nexus. Debe cumplir con la definición de terminado. Se inspecciona en la Nexus Sprint Review pero puede entregarse antes de la misma.

→ El compromiso de este artefacto es la Definición de Terminado, que define el estado del trabajo integrado cuando cumple con la calidad y las medidas requeridas para el producto. Es responsabilidad del Nexus Integration team una definición de terminado que se pueda aplicar en cada sprint, y todos los Scrum Team deben definir y adherirse a esta definición.

Cada Scrum Team se autogestiona para llegar a este estado, pudiendo aplicar una definición de terminado más estricta pero nunca usar criterios menos rigurosos de lo acordado para el Integrated Increment.

Concepto	Scrum 2020	Nexus (Escala)
Enfoque	Un solo equipo	Varios equipos (3–9) trabajando juntos
Meta de producto	Introduce Product Goal	Se mantiene una sola meta global
Roles	PO, Scrum Master, Developers	+ Nexus Integration Team
Artefactos	Product Backlog, Sprint Backlog, Increment	+ Refined Nexus Product Backlog
Eventos	Sprint Planning, Daily, Review, Retrospective	+ Nexus-level versions de esos eventos

Métricas en los 3 enfoques

¿Qué es una Métrica?

Una métrica es un número que representa el grado o presencia de un determinado conjunto de atributos respecto de lo que se quiere medir (proceso, producto o proyecto), expresadas de tal forma que permitan una medición objetiva de la realidad.

Por ejemplo, la escala: NS, S, B, MB, E no sirve. Lo que sí sirve es cuántos E hay, cuántos S hay, etc. **Debe ser posible medir en términos de la definición de esta y del esfuerzo que se debe aplicar para medirla.** Es decir que la métrica no es la escala sino la cantidad de cierta escala.

Todas las métricas poseen un **costo asociado**, debido a que se deben destinar recursos para su planificación, ejecución y análisis, y no siempre es factible disponer de esos recursos en un proyecto. Esto implica un **análisis de costo-beneficio** para evitar definir métricas que no aporten un valor a la organización/equipo/proyecto/etc.

En ese análisis costo-beneficio, **también influye la relación precisión-medición**, ya que muchas veces no es necesaria tanta precisión. Esto también quiere decir que las métricas no sólo hay que calcularlas y dejarlas, hay que utilizarlas para que justifiquen ese costo.

En el software es complicado tener métricas de algunas características del producto, como por ejemplo la usabilidad del producto, por lo que no se mide directamente al producto, sino que se utilizan conceptos asociados (epifenómenos) a la característica que se desea medir.

- Por ejemplo, si se quiere medir el uso de una funcionalidad, se puede medir a través de la cantidad de clicks en el botón que permite ejecutar esa funcionalidad.
- Por ejemplo, una métrica que dice cantidad de requerimientos tomado en función de la cantidad de requerimientos que deberíamos tomar, no es una métrica válida.

Las métricas son utilizadas para **dar visibilidad y tomar decisiones informadas de forma cuantitativa.**

¿Para qué medir?

- Para señalar, controlar y supervisar el desarrollo del proyecto.
- Para predecir al estimar proyectos de software.
- Para evaluar, es decir, para tener una noción de los costos, por ejemplo.
- Para mejorar el proceso, el proyecto o el producto.

Dominio de Métricas

1. Métricas de Proceso: son métricas estratégicas a nivel organizacional, tienen como objetivo **generar indicadores que permitan mejorar los procesos de software a largo plazo**. Son públicas y se busca obtener una métrica sobre el proceso de la organización en general, como un todo.
Son responsabilidad del **Ingeniero de Procesos**, quien realiza las mediciones y luego publica los datos para que todos los empleados de la organización puedan acceder a ellos.
 - Por ejemplo, se toman métricas de cantidad de defectos en todos los productos que se realizaron, se despersonalizan, se promedian a cada una de ellas y se publican a

toda la organización sin hablar particularmente de un proyecto o producto, sino en forma general al proceso de la organización.

Los indicadores de proceso permiten tener una visión profunda de la eficacia de un proceso.

LOS PROCESOS SON TRANSVERSALES A LA ORGANIZACIÓN!!

Ejemplos:

- Desviación organizacional al estimar.
- Defectos por severidad en productos de la organización.
- Errores previos a releases por proyecto.
- Defectos detectados por usuarios.
- Esfuerzo realizados.
- Tiempos de planificación promedio por proyectos.
- Propagación de errores de fase a fase.

2. Métricas de Proyecto: están enfocadas a los recursos que se dedican al proyecto, como costos, esfuerzos, estimaciones y tiempo. Son responsabilidad del **Líder de Proyecto** y permiten al equipo adaptar el desarrollo de los proyectos y de las actividades técnicas. Estas métricas son privadas de ese proyecto y solo son visibles para los involucrados en el mismo.

- **Se utilizan para mejorar la planificación del desarrollo**, generando ajustes que eviten retrasos, reduzcan riesgos potenciales y por lo tanto, problemas.
- Además, **se utilizan para evaluar la calidad de los productos en todo momento** y en caso de ser necesario, modificar el enfoque para mejorar la calidad, minimizando defectos, retrabajo y por ende el costo total del proyecto.

Las métricas de proyecto se consolidan con el fin de crear métricas de procesos que sean públicas para la organización de software como un todo.

Ejemplos:

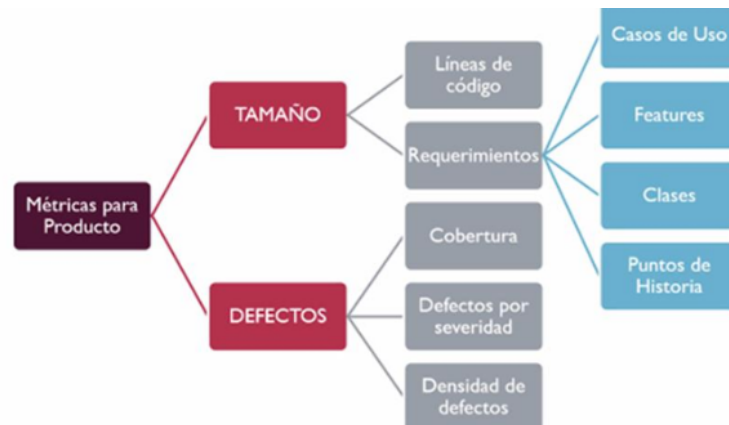
- Eficiencia de estimación del proyecto.
- Costos estimados versus costos reales.
- Esfuerzo/Tiempo por tarea del Ingeniero de Software.
- Errores no cubiertos por hora de revisión.
- Fechas de entregas reales versus programadas.
- Cantidad de cambios y sus características.

3. Métricas de Productos: están enfocadas en lo que se construye, son responsabilidad **del equipo de desarrollo y de Testing** y son particulares de ese producto.

Tienen como objetivo generar indicadores en tiempo real de la eficacia del análisis, el diseño, la estructura del código, la efectividad de los casos de prueba y calidad del software a construir.

Se deben controlar los artefactos resultantes del proceso de desarrollo (componentes y modelos) para garantizar:

- Que cumplan con los requerimientos del cliente.
- Que cumplan con los requerimientos de calidad.
- Que estén libres de errores.
- Que se realizaron bajo los procedimientos de calidad.



- Cobertura: tratar de que sea la mayor posible para cubrir el 100% del análisis de los defectos del producto.
- Defectos por severidad: cantidad de defectos por escala de severidad de cada defecto. 5 defectos graves.
- Densidad de defectos: cuantos defectos se encuentran por historia de usuario.

Tener en cuenta: Las métricas del proyecto, al ser recolectada sistemáticamente en múltiples iniciativas, pueden servir como insumo para definir métricas de proceso a nivel organizacional. De esta forma, la organización puede transformar datos específicos de proyectos de conocimiento reutilizables.

Métricas de proceso	Métricas de proyecto	Métricas de producto
Públicas	Privadas	Privadas
Ingeniero de procesos	Líder de proyecto	Equipo de desarrollo y de Testing

Métricas en el Enfoque Tradicional

En el enfoque tradicional se hace énfasis en los **3 dominios** arriba mencionados. Están basadas en la gestión de proyectos definidos.

Cuando se planifican los proyectos, allí se definen qué métricas se utilizarán, quiénes serán responsables de estas, cómo se calcularán, etc. No todas las métricas le interesan a todo el mundo, dependiendo del momento y rol es lo que interesa medir.

Los perfiles más técnicos apuntan a cuestiones relacionadas con el producto y esfuerzo (porque son ahí donde los desarrolladores asumen compromiso) y en ámbitos organizacionales interesa plata (costo) y tiempo. Por ejemplo:

- Testers: les interesa el esfuerzo y cantidad de defectos encontrados principalmente, porque es lo que más afecta a este rol.
- Líder de proyecto: El enfoque de las métricas estará más relacionado al calendario, los costos, plazos de tiempo a cumplir, etc.

Métricas básicas para un proyecto de software: son las mínimas e imprescindibles que uno tiene que utilizar si tiene la intención de desarrollar una cultura de medición:

- **Tamaño del producto:** asociada a métrica de producto.
- **Esfuerzo:** asociada a métrica de proyecto.

- **Calendario:** asociada a métrica de proyecto.
- **Defectos:** asociada a métrica de producto.

Estas métricas básicas permiten gestionar la triple restricción: alcance, tiempo y costo. En ambientes tradicionales las métricas utilizadas en proyectos anteriores suelen extrapolarse a nuevos proyectos. Esto se debe a que en el enfoque tradicional se busca la repetibilidad de los procesos.

Métricas en Ambientes Ágiles

Las métricas sirven para un equipo y **no son extrapolables**. En ágil hay un principio que habla específicamente de métricas (la mejor métrica de progreso es el software funcionando). Este principio apunta a que **vamos a medir producto** (no proceso ni proyecto). Solo se debe medir lo que sea necesario y nada más, es decir, lo que agregue valor para el cliente.

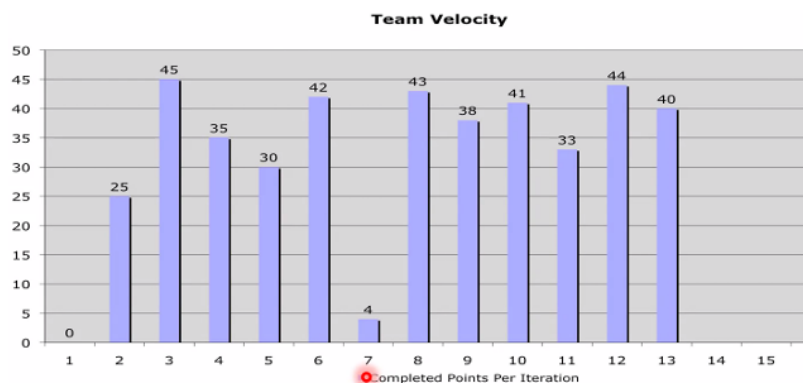
Hay dos principios Ágiles que guían la elección de las métricas:

- **El software funcionando es la principal medida del progreso:** Este principio apunta a que vamos a medir producto no proceso ni proyecto).
- **La mayor prioridad es satisfacer al cliente por medio de entregas tempranas y continuas de Software valioso y funcionando:** Esto apunta a que solo se mide lo estrictamente necesario y nada más, es decir, lo que agregue valor para el cliente.

Métrica de Velocidad (cantidad de SP terminados y aceptados por el PO)

Es la métrica más importante del enfoque, y mide la cantidad de puntos de historia que se realizaron en un Sprint y fueron aceptados por el Product Owner. Osea las US que pasaron el Definition of Done. Esta métrica se calcula (no se estima) al final de un sprint/ iteración.

Esta métrica se suele representar con un gráfico de barras para medir la estabilidad del equipo, lo que también se relaciona con un principio del manifiesto ágil (desarrollo sostenible).



Métricas de Capacidad (cantidad de horas de trabajo ideales o StoryPoints —> compromiso del equipo)

Es una métrica de proyecto que **mide el compromiso de un equipo para un determinado sprint, en horas de trabajo ideales**. Es por esto que, se utiliza para planificar, ya que permite definir cuántas historias de usuario se van a tomar del Product Backlog para implementar en el próximo sprint.

Se estima al principio de un sprint, por lo que **también se la llama velocidad estimada**. Se puede estimar en horas ideales si el equipo es principiante o en Story Points si el equipo es maduro.

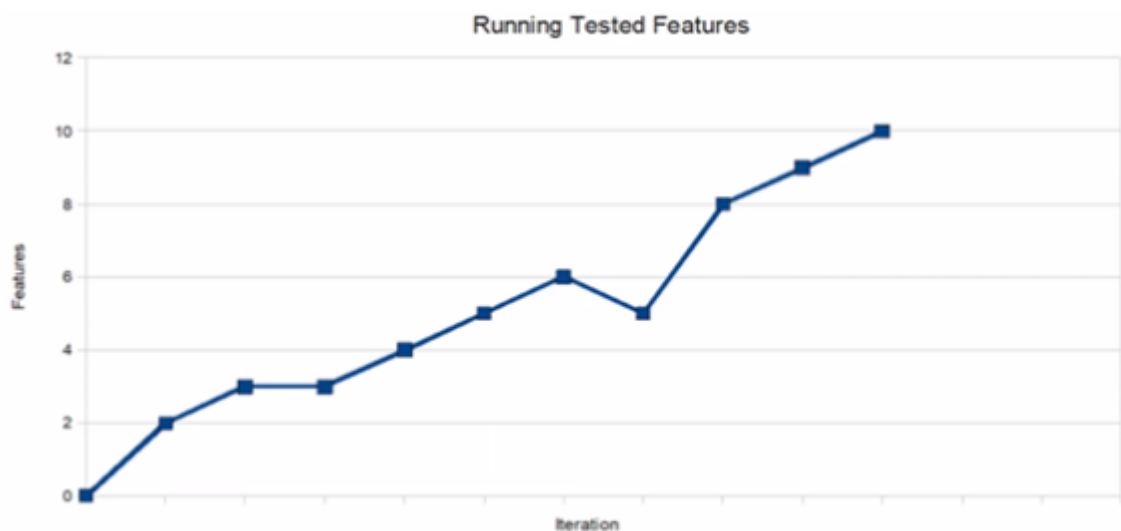
Es una estimación de cuánto trabajo (puntos de historia) va a poder completar el equipo para un determinado sprint, en horas de trabajo ideales.

Running Tested Features (RTF) (cantidad de características del producto terminadas y en ejecución)

Mide la cantidad de features testeadas que están funcionando, es decir, cuantas piezas de producto (historias de usuarios, casos de usos, requerimientos) se terminaron y están en ejecución.

Al medir solo la cantidad, NO se considera el tamaño o complejidad de cada una. Por lo que es recomendable complementarla con otras métricas para obtener una evaluación más completa **del progreso y valor entregado**.

Por ejemplo, si se toma como piezas de producto a historias de usuario, se mide cuántas historias de usuarios están funcionando, pero no se sabe de cuantos puntos de historias tiene cada una de ellas.



Si la curva es constante (llana) o tiene pendiente negativa, entonces indica la existencia de un problema, ya que las características no incrementan o disminuyen en el tiempo. La pendiente es negativa cuando una funcionalidad del sistema deja de ser válida.

Métricas de software en Lean (KANBAN)

El foco de medición de Kanban **es el proceso**, ya que el sistema de trabajo de Kanban es introducir mejoras en un flujo de trabajo continuo, es decir, **no existe un proyecto con principio y fin**. Las métricas ayudan a identificar cuellos de botella y oportunidades de mejora.

Lead Time o Elapsed Time

Métrica de vista o perspectiva del cliente. Es la más importante para el cliente. Mide desde el momento en que el cliente me pide algo (entra al Backlog) hasta que yo se lo entregó.

Cycle Time

Mide el tiempo desde que el equipo comenzó a trabajar sobre una funcionalidad pedida, hasta que se lo entrega, **eliminando el tiempo de espera en el Backlog**. Sirve al equipo de trabajo y no es tan relevante para el cliente.

Touch Time

Mide los tiempos en los cuales las personas están efectivamente trabajando sobre una unidad de trabajo (columnas de producción), **sin tener en cuenta aquellos tiempos de espera (columnas de acumulación).**

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

Eficiencia del ciclo de proceso

Esta métrica mide la eficiencia del tiempo para entregar un trabajo, respecto al tiempo destinado a su ejecución. Se calcula como Touch time / Lead time.

Mientras más cercano a 1 sea su valor, más eficiente es el proceso, ya que todo el tiempo se estuvo trabajando sobre alguna funcionalidad, y no hubo muchos momentos de espera en columnas de acumulación.

Métricas tradicionales	Métricas ágiles	Métricas lean
• Proceso • Proyecto • Producto	• Producto	• Proceso
Son extrapolables Para métricas de Proceso: Se despersonalizan las métricas de proyecto y producto	NO son extrapolables No se despersonalizan	
★ Tamaño → Producto ★ Esfuerzo → Proyecto ★ Calendario → Proyecto ★ Defectos → Producto	★ Velocidad ★ Capacidad ★ Running tested features	★ Lead Time o Elapsed Time ★ Cycle Time ★ Touch Time ★ Eficiencia del ciclo de proceso

UNIDAD 3: Gestión de Software Como Producto

Un producto de Software es un artefacto que se construye para satisfacer una necesidad específica en base a ciertos requerimientos.

La gestión de configuración busca responder a los cambios y mantener la integridad del producto y garantizar su calidad. Trasciende el proyecto ya que se enfoca en el producto. Para esto adopta prácticas continuas.

Prácticas Continuas

Integración Continua (Continuous Integration)

- Es una práctica de desarrollo que **promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día. Cada integración de código es luego verificada por una serie de pruebas automatizadas**, permitiéndole al equipo detectar problemas de manera temprana.
- La integración continua busca **asegurar que el software pueda ser desplegado en cualquier momento**, es decir, que el código compile y que sea de calidad.
- Cada desarrollador en su entorno de trabajo realiza pruebas unitarias (en la medida de lo posible, automatizadas) **desarrollando con TDD** (desarrollo conducido por pruebas) y cuando termina el componente de código, lo prueba y sabe que funciona, lo sube a un repositorio de integración.

Entrega Continua (Continuous Delivery)

- Es una disciplina de desarrollo de software en la que **el software se construye de tal manera que puede ser liberado en producción en cualquier momento**. Suma la **automatización de las pruebas de aceptación**, una vez pasadas estas el producto ya está listo para desplegarlo a producción.
- No implica necesariamente que se libere cada vez que hay un cambio, sólo ocurre si el responsable de negocio, el Product Owner de turno, decide pasar a producción.
- **La integración continua es prerequisite de la entrega continua**. Los artefactos producidos en el servidor de integración continua son puestos en producción con solo un click.
- Hay que asegurarse de tener un **despliegue automatizado**, para que cada puesta en producción no lleve demasiado tiempo, de manera que puedan hacerse más seguidas y que cada despliegue no introducir demasiados cambios juntos.
- **El software debe estar siempre en un estado de “entregable”**.

Despliegue Continuo (Continuous Deployment)

- Consiste en **poner el producto en el ambiente de producción del usuario final**.
- A diferencia de la entrega continua, **en el despliegue continuo no existe la intervención humana para desplegar el producto en producción**. Para esto, se utilizan pipelines que contienen una serie de pasos que deben ejecutarse en un orden determinado para que la instalación sea satisfactoria.
- El propósito es que sea transparente para el usuario que pusiste en producción una nueva versión del producto.
- Se recomienda hacer el despliegue a poco tiempo de haber trabajado con los cambios en el código

Tenemos diferentes Estrategias de despliegue continuo:

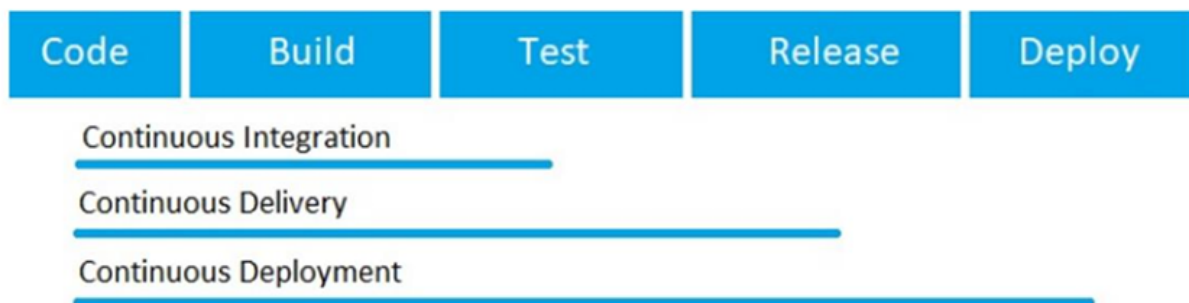
Canary Deployment

Es la práctica de **realizar lanzamientos por etapas**. Primero implementamos una actualización de software **para una parte de los usuarios**, para que la prueben y proporcionen comentarios. **Una vez aceptado el cambio, la actualización se extiende al resto de usuarios**. Esta estrategia nos muestra cómo los usuarios interactúan con los cambios de la aplicación en el mundo real.

Blue-Green Deployment

Es un modelo de lanzamiento de aplicaciones que **transfiere gradualmente el tráfico de usuarios de una versión anterior** de una aplicación o microservicio **a una nueva versión casi idéntica**.

La versión anterior puede denominarse **entorno azul**, mientras que la nueva versión puede denominarse **entorno verde**. Una vez que el tráfico de producción se transfiere por completo de azul a verde, el azul puede quedar en espera en caso de reversión o retirada de producción.



La diferencia entre estas tres prácticas es el nivel de automatización de las pruebas.

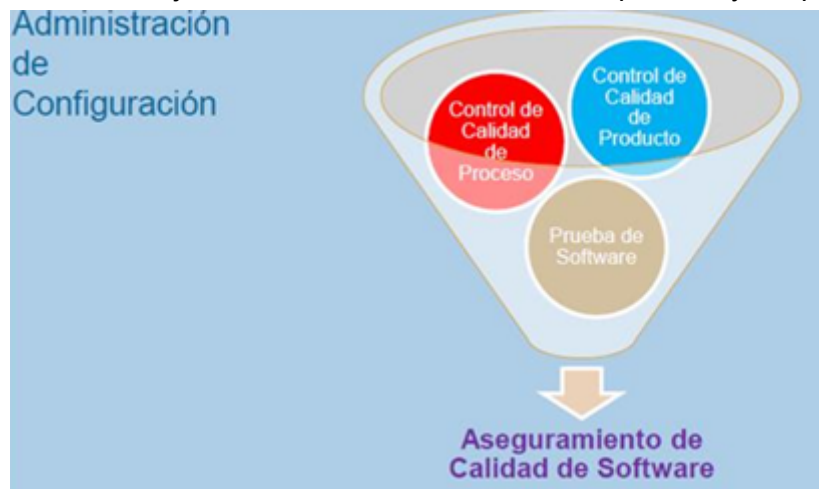
UNIDAD 4: Aseguramiento de calidad de Proceso y de Producto

La **calidad** es el cumplimiento de los requerimientos funcionales y de performance explícitamente definidos, de los estándares de desarrollo explícitamente documentados y de las características implícitas esperadas del desarrollo de software profesional.

Testing en ambientes Ágiles

Contexto

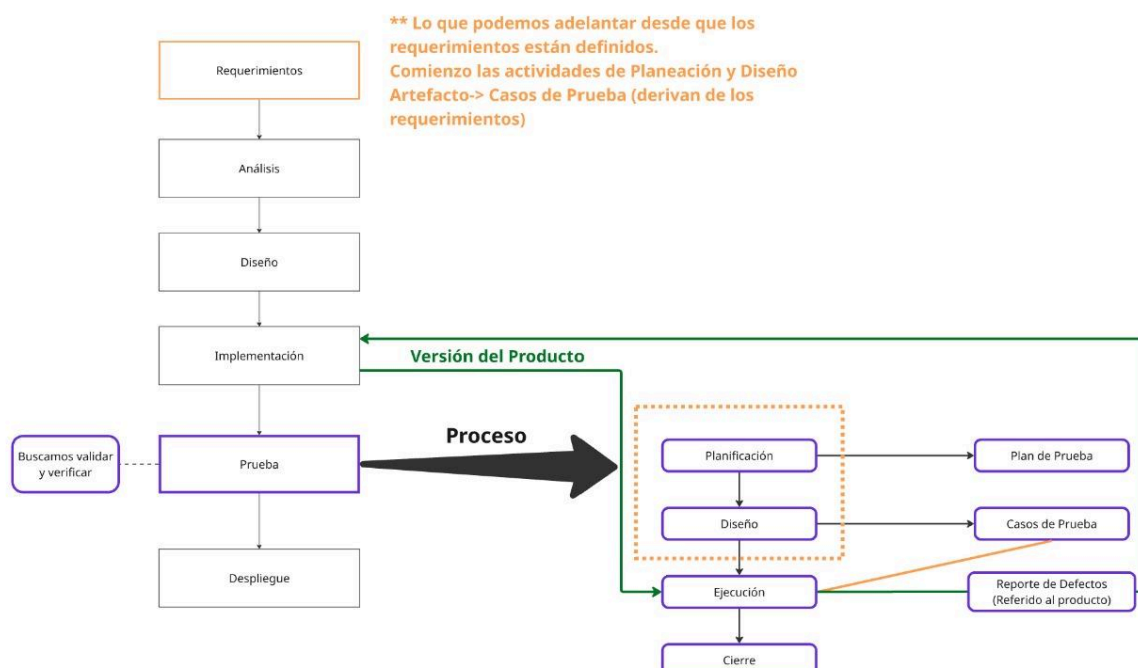
El testing de software, es una de las tareas a realizar en el ámbito del aseguramiento de calidad de software, en conjunto con el control de calidad del proceso y del producto.



La calidad del software se obtiene y se logra a lo largo de todo el desarrollo de este, por lo que una buena administración de configuración (SCM) es el puntapié inicial para permitir las tareas de aseguramiento de calidad. **La calidad del producto depende de la calidad del proceso que se está utilizando.**

El Testing es parte del QA (Quality Assurance). Se hace cuando el producto ya está construido, en cambio el QA se aplica en todo el ciclo de vida.

¿Donde esta el testing en el PUD?



Definición de Testing

Proceso mediante el cual se somete a un software a condiciones específicas para determinar si el mismo es válido o no **en función de los requerimientos especificados**.

Es el proceso DESTRUCTIVO de tratar de encontrar defectos en el código para lo que asumimos que en el software hay defectos.

No debemos pensar que no hay defectos ni que debemos verificar que no los hay, sino que empezamos con la hipótesis o la actitud negativa de que el software es incorrecto.

El testing exitoso es el que encuentra defectos. Un desarrollo exitoso nos conduce a un test no exitoso.

En el desarrollo de un software confiable, el testing se lleva entre un 30% y 50% del costo total del mismo.

Error vs Defecto vs Falla

Tanto el **error** como el **defecto** pueden provocar una **falla** en el sistema. Si no hay mal funcionamiento del sistema entonces no hay falla.

- Un **error** es algo que se origina y se descubre en la misma etapa en la que se está trabajando. Es decir, es un error cuando no se traslada a otras etapas, se detecta y se corrige en la misma.
- Un **defecto** es un error que se trasladó de etapas. Nos demuestran un error no detectado que se trasladó a una etapa siguiente.

En el testing encontramos defectos ya que justamente se encuentran cosas incorrectas que se realizaron **en la etapa de implementación que es la etapa anterior**.

Es mejor encontrar errores antes que defectos.

- Métrica: edad del defecto → tiempo desde que se introdujo el defecto hasta que se detectó

Severidad y Prioridad

Cuando hablamos de defectos encontrados durante el testing, debemos considerar dos aspectos que nos van a definir el accionar que vamos a realizar sobre estos:

- **SEVERIDAD:** Es que tan grave es el defecto encontrado. Es decir el impacto que produce en el sistema. La define el equipo.
 - **Bloqueante o invalidante:** No se puede usar el sistema, no anda
 - **Crítico**
 - **Mayor**
 - **Menor**
 - **Cosmético:** Suelen ser atribuidos a UX como errores de ortografía etc.
 - **Mejora:** *no es un defecto, no es algo que está mal. Es algo que al testing le parece que se podría mejorar.*
- **PRIORIDAD:** Es la urgencia que tenemos para resolver este defecto. La asigna el cliente.
 - Urgencia
 - Alta
 - Media
 - Baja

Se podría intuir que, según la severidad de los defectos, va a haber tal o cual prioridad, pero esto no siempre es así. Esto va a depender del contexto en el cual nos encontramos, es por esto la importancia de tener los dos aspectos identificados.

Por ejemplo: que los números se muestran: 1000000000 es un **error cosmético**, no hay separadores de miles, pero según el contexto puede tener **prioridad urgente** ya que directamente no se puede trabajar así.

Niveles de Prueba

Los niveles de prueba determinan la granularidad de la prueba que se está por ejecutar. En general, primero se comienzan probando componentes pequeños, y luego se integran en pruebas de mayor granularidad.

- **PRUEBAS UNITARIAS** (en la implementación): las hace el desarrollador y están incluidas en el DoD. Son pruebas que **se realizan sobre un componente**, de manera independiente a los otros. Se hacen teniendo acceso al código fuente, y se pueden usar herramientas para realizarlas (automatización, depuración). Se suelen reparar los errores apenas se encuentran, sin registrarlos formalmente.

Estas son las pruebas que se suelen aplicar en el TDD

No busca encontrar defectos (se hace en la misma etapa de implementación) y no busca encontrar errores

- **PRUEBAS DE INTEGRACIÓN** (en el desarrollo o testing): Se verifica si los componentes que pasaron las pruebas unitarias funcionan bien juntos. Se prueba el funcionamiento conjunto de clases en las cuales existe una interacción a modo de peticiones, a través de sus **interfaces (boundaries)**.

Para identificar bien los errores se lleva a cabo de manera incremental, desde lo más general a lo más específico (top-down) o desde lo más específico a lo más general (bottom-up).

Si busca defectos

- **PRUEBAS DE SISTEMA (O VERSIÓN)** (prueba): En estas pruebas se realizan testeos sobre la versión de un incremento de producto o de un producto. Si o si las realizan personas ajenas al desarrollo de los programas (tester). Estas pruebas se llevan adelante siguiendo un proceso sistemático, que permite encontrar **defectos que puedan ser reproducibles** y cual es el camino de pasos que hay que seguir para encontrarlos.

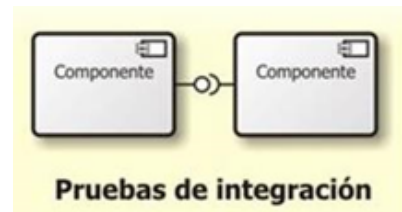
Abarca requerimientos funcionales y no funcionales.

Estas pruebas están **basadas en casos de prueba** que prueban el funcionamiento real y cotidiano del producto.

Se trata de emular un entorno de trabajo idéntico al entorno real en el que se usará el SW.

Si busca defectos

- **PRUEBAS DE ACEPTACIÓN DE USUARIO** (en el despliegue): Se realizan en el despliegue y debería realizarlas el usuario para verificar que se cumple con lo que el mismo requirió. Busca que el cliente/usuario se familiarice con el producto, generando comodidad y confianza sobre el mismo (su objetivo principal no es encontrar fallas). Comprende



- tanto la prueba realizada por el usuario en ambiente de laboratorio (pruebas alfa),
- como la prueba en ambientes de trabajo reales (pruebas beta). En la teoría, los usuarios arman sus pruebas de usuario.

No busca defectos, sino una capacitación sobre la nueva funcionalidad.

Ambientes para la construcción del software

Son todos los recursos (hardware y software) que se requieren para trabajar el producto.



De MINIMO necesitamos

- Desarrollo
- Prueba
- Producción

Muchas veces no se arman ambientes de pre-producción porque es costoso armar un entorno con los mismos recursos que hay en el ambiente de producción.

Igual deberíamos tenerlo ya que **probar en producción es muy riesgoso.**

- **DESARROLLO:** El ambiente de desarrollo es donde los desarrolladores crean, prueban y depuran el software. Es un entorno local o en red utilizado por los desarrolladores para escribir y probar el código antes de integrarlo con el resto del sistema. Este ambiente permite a los desarrolladores experimentar y probar nuevas ideas sin afectar los entornos de producción. Las **pruebas unitarias** se llevan a cabo en el ambiente de desarrollo
Implica hardware y software necesarios (librerías, extensiones, IDEs, compiladores, etc.) para poder desarrollar y desplegar el producto y utilizarlo.
- **PRUEBAS:** El ambiente de pruebas es donde se llevan a cabo las pruebas exhaustivas del software desarrollado. Aquí se realizan pruebas de integración, pruebas funcionales, pruebas de rendimiento y otras pruebas relevantes para verificar que el software cumpla con los requisitos establecidos y funcione correctamente. Las **pruebas de integración y de sistema** se realizan en el ambiente de pruebas
- **PRE-PRODUCCIÓN:** El ambiente de preproducción, es donde se realiza una prueba final del software antes de su lanzamiento en producción. Aquí se simulan las condiciones del entorno de producción, pero sin afectar a los usuarios finales y sin TODAS las características (ya que es caro tener ambientes iguales al de producción). Se utilizan **conjuntos de datos** y configuraciones similares a las del entorno de producción para garantizar una prueba más precisa del software, y se realizan pruebas de último minuto, como pruebas de estrés, pruebas de carga y pruebas de seguridad. El objetivo es validar y verificar que el software esté listo para ser implementado en el entorno de producción sin problemas significativos. Las **pruebas de aceptación** se llevan a cabo en el ambiente de preproducción
- **PRODUCCIÓN:** El ambiente de producción es donde el software está en funcionamiento y es **accesible a los usuarios finales**. Es el entorno real en el que

el software se utiliza para realizar las tareas y funciones para las que fue diseñado. Este ambiente suele ser altamente controlado. Se implementan medidas de respaldo y recuperación ante desastres para garantizar la disponibilidad continua del software. Obviamente **en este entorno no se realizan pruebas** (probar en este ambiente tiene grandes consecuencias), ya que en teoría la versión del producto cumple con los criterios del Definition of Done.

Cuando probamos, ¿Qué probamos? Tipos de pruebas

Probamos los requerimientos funcionales y los no funcionales.

Por lo general se prueban más los RF ya que muchas veces no se arman los entornos iguales al de producción necesarios para probar los RNF.

Tipos de pruebas

1. **TESTING FUNCIONAL:** Las pruebas se basan en funciones y características (descrita en los documentos o entendidas por los testers) y su interoperabilidad con sistemas específicos. Utilizan los requisitos definidos en una ERS o los acuerdos que contienen las pruebas de usuario y los criterios de aceptación de una US para realizar las pruebas.
 - a. **Pruebas de requerimientos en operación normal:** apunta a probar una funcionalidad sola de manera que PASE.
 - b. **Pruebas de requerimientos negativos:** es usar mal el sistema a propósito para que la prueba FALLE.
 - c. **Proceso de negocio:** cuando se prueba un proceso de negocio completo, es decir, se prueba todo el proceso. Por ejemplo, en una venta se prueba la búsqueda del artículo, la selección y facturación del mismo.
 - d. **Documentación:** es importante destacar que la documentación no tiene que ser si o si un documento tipo word sino que puede estar integrado al sistema, ser un video, etc. En este tipo de prueba se ve por ejemplo:
 - i. que el manual de usuario muestre lo que el sistema hace y cómo el usuario puede alcanzar sus objetivos usando el sistema.
 - ii. que el manual de configuración, que muestre como realmente se configura el sistema.
2. **TESTING NO FUNCIONAL:** Se basa en cómo trabaja el sistema haciendo foco en los requerimientos no funcionales (foco en el “cómo”, no en el “que”). Son las pruebas más complejas, por su gran dependencia al entorno del sistema, por lo que este debe ser lo más parecido posible al del cliente. Sin embargo, se tienen características que no pueden probarse, como el ancho de banda del internet, la seguridad o performance en una determinada situación, que se pueden solucionar con las pruebas de aceptación. Incluye varios tipos de prueba como:
 - a. **Performance:** Se verifica si el tiempo de respuesta está dentro de lo estipulado. Tiene que estar ligado a **conurrencia**
 - b. **Interfaces:** Se prueba con las interfaces de usuarios. UX Es imposible cubrir el 100% así que se acuerda con el PO o el cliente a la hora de diseñar los casos de prueba.
 - c. **Escala completa:** Que ande todo a lo que me comprometí.
 - d. **Estrés:** Queremos forzar al sistema para que falle, se lo somete a condiciones más allá de las normales. Se ve el tiempo de recuperación del mismo. El objetivo es ver que pasa cuando el sistema se cae.
 - e. **Seguridad:** Se ataca al sistema con hackeos éticos.

- f. **Accesibilidad:** Que funcionen los mecanismos de accesibilidad para los usuarios, sobre todo para usuarios con discapacidades.
- g. **Portabilidad:** se prueba en los distintos entornos acordados con el cliente.
- h. **Carga:** no solo mira performance, mira el comportamiento de los dispositivos de hardware (procesadores, discos, etc.) y de las comunicaciones.
- i. **Mantenimiento:** para ver si el producto está en condiciones de evolucionar, se controla que haya documentación, manual de configuración, etc. Se observa la facilidad que existe para corregir un defecto.
- j. Etc hay muchísimas

Todas estas pruebas se pueden hacer a nivel de sistema y de aceptación pero no todas se pueden hacer a nivel unitario.

3. **SMOKE TEST** Es un tipo de prueba que se hace para validar que no haya fallas groseras, de gran magnitud, en el producto de software. Se realiza **antes de comenzar el ciclo 0**. Nos ahorra empezar a testear formalmente si encuentra un error, porque todavía hay algo que corregir. Consiste en una corrida rápida para ver si el producto está en condiciones de pasar al ciclo de prueba.

Casos de prueba

Un caso de prueba es una situación descrita y detallada que apunta a verificar y validar una función del sistema.

El caso de prueba ejecuta una serie de pasos o acciones en una determinada funcionalidad, determinando completamente con qué valores se va a hacer la ejecución para ver si el software me va a dar los resultados esperados o no.

Los casos de prueba salen de los requerimientos, permitiendo hacer tanto la verificación como la validación.

La buena definición de casos de prueba nos ayuda a reproducir defectos.

Están compuesto por:

- **Un objetivo:** la característica del sistema a comprobar
- **El escenario:** el paso a paso del caso de prueba
- **Pre condiciones/ condiciones de seteo /DATOS DE SETEO?:** todos los datos que necesito disponibles para llevar a cabo el escenario.
- **Resultado esperado:** la salida o la acción esperada en el sistema de acuerdo a los requerimientos del mismo

Las pruebas deben ser detalladas, con datos concretos. El éxito de las pruebas está muy relacionado con los datos usados.

La idea es con la menor cantidad de casos de prueba encontrar la mayor cantidad de defectos posibles.

Para esto vamos a usar **estrategias de prueba**, con estas estrategias vamos a diseñar los casos de prueba de manera que conduzcamos el esfuerzo hacia lo importante.

Condición de prueba

Tienen que ver con el contexto del sistema que tengo que tener en cuenta para poder hacer los casos de prueba. Esta es la reacción esperada de un sistema frente a un estímulo particular, este estímulo está constituido por las distintas entradas. Es algo que se enuncia ANTES de definir los casos de prueba, defino las condiciones y en función de eso defino los casos. Una condición de prueba debe ser probada por al menos un caso de prueba.

Estrategias de prueba

- Caja negra: En esta estrategia no se dispone de la estructura interna de la implementación, o sea no puedo ver el código, analizo las funcionalidades en términos de entradas y salidas de una “caja negra”.

El proceso consiste en ingresar determinados datos a esa funcionalidad vista como caja negra, y luego comparar los resultados obtenidos con los resultados esperados.

- **Métodos basados en especificaciones:** Son aquellos que se ejecutan utilizando la documentación de especificaciones realizadas del producto.
 - Partición de equivalencias: proceso sistemático que consiste en identificar clases de equivalencia. Estas son subconjuntos de valores posibles que producen un resultado equivalente.
 - Análisis de valores límites: variante del anterior. Utilizar los límites o valores de borde de las clases de equivalencia para la definición de los casos de prueba.
- **Métodos basados en experiencia:** utiliza la experiencia y los conocimientos del tester para determinar las entradas del sistema y analizar los resultados.
 - Adivinanza de defectos: el tester identifica pruebas que probablemente expongan defectos del software, elaborando una lista de defectos posibles y realizando pruebas a partir de esa lista.
 - Testing exploratorio: el tester va probando el software, mientras va aprendiendo a manejar el sistema y junto con su experiencia y creatividad, genera nuevas pruebas a ejecutar.
- Caja blanca: En estos se dispone del código (puede ser también pseudocódigo o un diagrama de flujo). Nos permiten diseñar casos de prueba, analizando la estructura interna del software o un componente del software. Hay distintos métodos.

Hay distintas coberturas con nivel diferente (la forma de recorrer los distintos caminos que provee el código para desarrollar una funcionalidad):

- **Cobertura de enunciados o caminos básicos:** ● Objetivo → encontrar todos los caminos independientes de una funcionalidad que deben recorrer (testear) al menos una vez, para probar cada uno de ellos con casos de prueba.

Permite obtener una métrica denominada *complejidad ciclomática (M)*, que representa la cantidad de caminos independientes que posee una funcionalidad, y el mínimo número de casos de prueba que hay que construir.

$$M = E - N + 2 \cdot P$$

- M = complejidad ciclomática.
- E = número de aristas del grafo.
- N = número de nodos del grafo.

- P = número de componentes conexos o nodos de salida.

O M = cantidad de regiones cerradas + 1

- **Cobertura de sentencias:** Sentencia: cualquier instrucción como asignación de variable, invocación de métodos, etc. • Objetivo → buscar la cantidad mínima de casos de pruebas que me permitan evaluar todas las sentencias.
- **Cobertura de decisión:** Decisión: estructura de control concreta, dentro de las cuales existen combinaciones unidas por operaciones booleanas de condiciones. • Objetivo → buscar la cantidad mínima de casos de prueba que me permitan evaluar que todas las ramas de las decisiones derivan en la rama del true y en la rama del false cuando corresponda en cada caso.
- **Cobertura de condición:** Condiciones: son cada una de las evaluaciones lógicas dentro de una decisión, que están unidas por operadores lógicos. • Objetivo → buscar la cantidad mínima de casos de pruebas que me permitan evaluar cada una de las condiciones, en su valor verdadero y en su valor falso.
- **Cobertura de decisión condición:** • Objetivo → evaluar las ramas verdaderas y falsas de las decisiones, y a su vez evaluar valores verdaderos y falsos de las condiciones.
- **Cobertura múltiple:** • Objetivo → evaluar el combinatorio de todas las condiciones, en todos sus valores de verdad posibles.

Dentro de las estrategias de caja negra tenemos las basadas en especificaciones → Particiones de equivalencia:

Particiones de equivalencia:

Analiza las condiciones externas (entradas y salidas) involucradas en una funcionalidad determinada. A esas condiciones externas las divide en clases de equivalencia, las cuales son subconjuntos de valores posibles, que arrojan resultados equivalentes en la funcionalidad que se quiere probar.

- Ejemplos de entradas: campos de texto, fechas, coordenadas de un mapa, archivos, señales de sensores, etc.
- Ejemplos de salidas: mensaje en pantalla, advertencias, listados, tablas, emisión de señal, etc.

Procedimiento:

1. Identificar condiciones externas de entrada y salida.

- a. Si hay que ingresar dos datos que son iguales, pero con distintos valores, separarlo en condiciones externas diferentes. Por ejemplo: 2 calles, una condición externa es calle1 y la otra calle2.

2. Identificar subconjuntos de valores posibles (válidos y no válidos) de las condiciones externas identificadas, que produzcan resultados equivalentes.

- a. Los subconjuntos no válidos tienen asociados mensajes de error en la condición externa de salida “Mensajes de Error” o “Mensajes de Advertencia” (no es necesario poner todos los mensajes, con algunos genéricos es suficiente).
3. **Armar los casos de prueba** tomando de cada condición externa, un sólo valor particular (especificar) de cada subconjunto de valores posibles.

Consideraciones:

- **Prioridad**
 - Alta → caminos felices o errores críticos en el dominio de negocio.
 - Baja → casos de prueba relacionados a validaciones (valores no ingresados, con mal formato, etc.).
- **Nombre caso de prueba:** describir el escenario de la funcionalidad que se está probando, ¡de forma clara! Por ejemplo: “Ingresar al sitio web con edad menor a 18 años”

Precondiciones:

Conjunto de características que tienen que cumplirse en el contexto de la funcionalidad, **para que pueda ser ejecutada.**

- Deben ser valores concretos, específicos. Por ej: “El usuario Juan está logueado con permiso de administrador.” - “La fecha actual es 25/10/2022” - “Las coordenadas del GPS son: 41°24’12.2” N” - “La tarjeta tiene el número: XXXX XXXX XXXX XXXX”
- Los datos que se seleccionan de entre un grupo determinado (por ejemplo forma de pago), son precondiciones que deben estar previamente cargadas en el sistema.

Pasos:

- Siempre arrancan seleccionando la opción que desencadena la funcionalidad: “El *nombreUsuario* ingresa a la opción ...”
- Después: “El *nombreUsuario* ingresa/selecciona ...”
 - No tiene que haber ambigüedad, pensar que otra persona los tiene que leer y ejecutar.
 - Normalmente finalizan con un botón de confirmación o algo así.

Resultado esperado

- Deben ser resultados concretos, específicos.
- Suelen ser mensajes de advertencia/error, listados, posición en el mapa, etc. pero siempre indicando con un dato particular. Por ejemplo: El sistema muestra el mensaje de advertencia “Debe seleccionar una fecha”.
- Se debe relacionar la salida con el paso en la cual se produce.

Ciclo de prueba o ciclo de test

Un ciclo de pruebas abarca la ejecución de la totalidad de los casos de prueba establecidos aplicados a una misma versión del sistema a probar.

Para correr un ciclo de prueba, osea correr los casos de prueba en el ambiente de prueba y obtener los resultados, necesito:

- La versión del producto
- los casos de prueba para la versión del producto

Idealmente se tienen 2 ciclos **por cada versión del producto**, el ciclo 0 y el 1er ciclo donde todo corre bien.

El primer ciclo el ciclo 0, este es siempre manual, es donde se configura todo y a partir del ciclo 1 ya se pueden automatizar las pruebas.

En caso de detectarse defectos, el producto vuelve a desarrollo para su corrección.

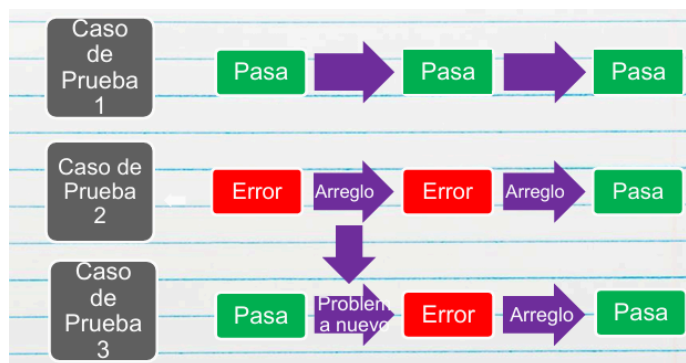
Tenemos 2 estrategias para probar cada ciclo de prueba

- **Con regresión:** Al concluir un ciclo de pruebas, y reemplazarse la versión del sistema sometido al mismo, debe realizarse una verificación total de la nueva versión, a fin de prevenir la introducción de nuevos defectos al intentar solucionar los detectados.

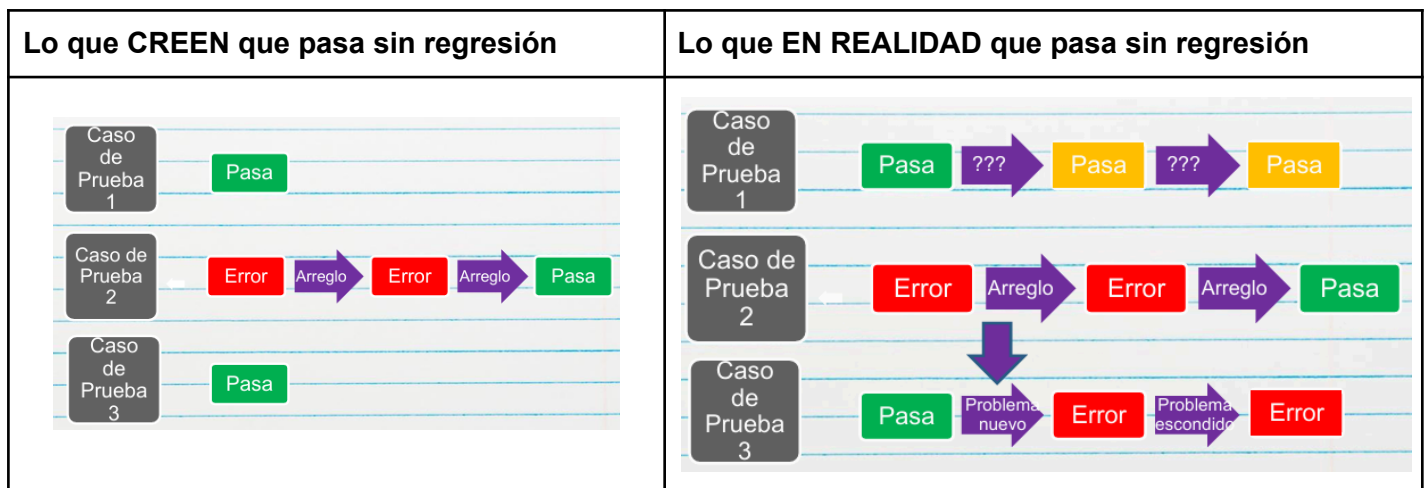
La teoría nos dice que cuando solucionamos un defecto normalmente se introducen 2/3 defectos más. Es importante volver a verificar la nueva versión **en su totalidad** para asegurarnos de no haber introducido nuevos defectos. No siempre debemos hacer regresión, eso lo vamos a ir viendo a medida que testeamos.

La automatización del testing permite hacer tantos ciclos de prueba con regresión como se deseen.

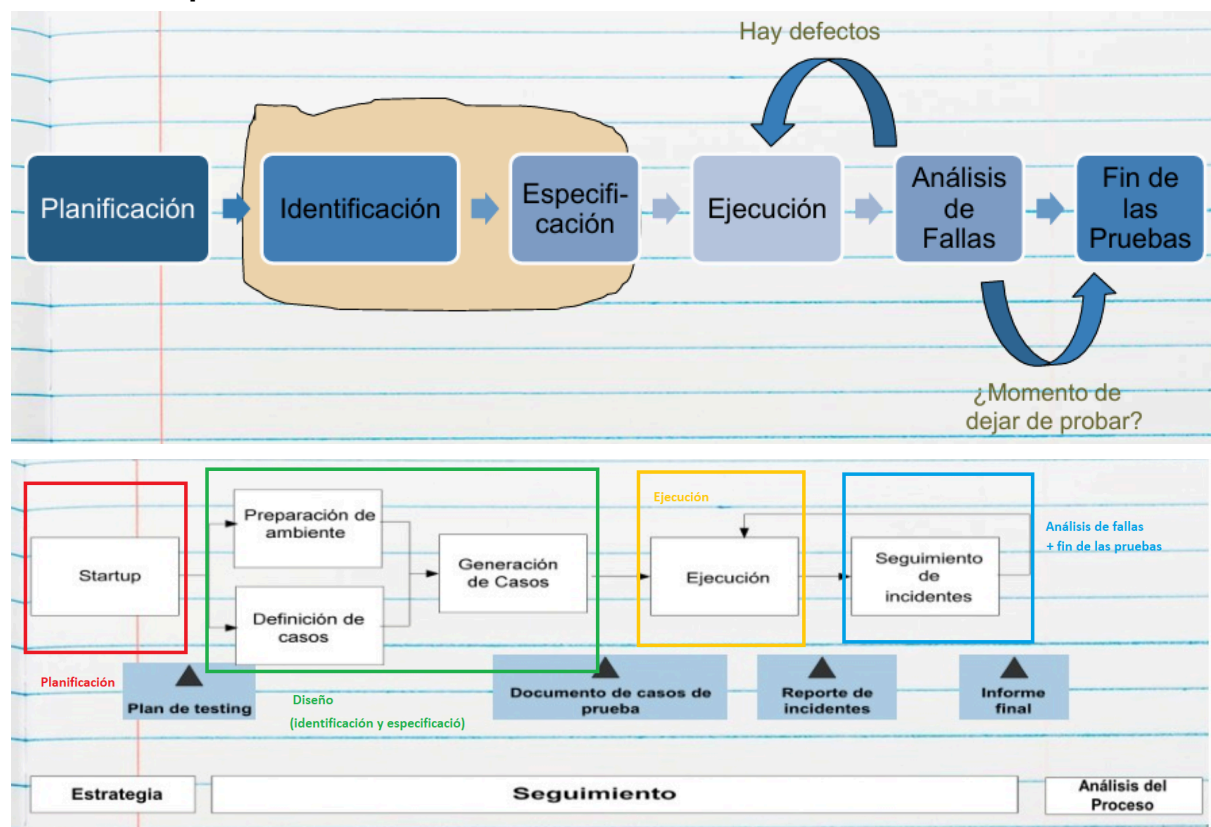
La desventaja de la regresión es el esfuerzo del proceso de automatizar la ejecución de los casos de prueba, pero se puede ver ese esfuerzo de automatización como el esfuerzo propio del ciclo 0 manual, y luego esa automatización permite ejecutar los casos de prueba N veces.



- **Sin regresión:** Al concluir un ciclo de pruebas, y reemplazarse la versión del sistema sometido al mismo, se vuelven a probar únicamente los casos de prueba que anteriormente no pasaban. Esta técnica no es recomendable ya que en la nueva versión al intentar corregir los defectos, se pueden haber introducido otros defectos ocultos en los casos de prueba que si pasaban.



Proceso de pruebas



- **Planificación:** Determina **cómo se incluirá el Testing en el plan del proyecto**, verifica que se entiendan las metas y objetivos del cliente y los stakeholders, y cómo será el Test Plan (que recursos se usará, qué riesgos se tendrá, cuál será el criterio de aceptación, qué entornos se emularon, quién realizará cada Testing, cuando, etc.). El resultado de la planificación es el Plan de Pruebas, que debe contener:
 - Riesgos y objetivos del Testing.
 - Estrategia de Testing.
 - Recursos.
 - Criterio de Aceptación.

- **Diseño (Identificación y especificación de casos de prueba):** Revisando las bases de la planificación del Testing, se identifican los datos necesarios, se diseñan y priorizan los casos de prueba que se llevarán a cabo y se diseña el entorno de prueba (se define que entorno se usara, como se llevaran a cabo, por quien, cuando, etc.). Analiza si los requerimientos son testeables o no. Se define si se usa regresión o no.
- **Ejecución:** Incluye la creación de los datos necesarios para la prueba, automatización de todo lo que sea necesario, implementar y verificar el ambiente, ejecutar los casos de prueba, registrar los resultados de la ejecución y comparar los resultados reales con los esperados.

Cuando se ejecutan los CP, se registran y se comparan los resultados generando un reporte de defectos

- **Análisis de fallas (Evaluación y Reporte):** Se hace un seguimiento de la corrección de los defectos encontrados hasta que se cierren todos los CP, es decir que se hayan solucionado todos. Se evalúa el criterio de aceptación, se reporta el resultado de las pruebas a los interesados, se verifica los entregables y que los defectos se hayan corregido y se evalúa cómo resultaron las actividades de testing y se analizan las lecciones aprendidas.
- **Fin de las pruebas:** En las empresas más maduras se deja de probar recién cuando no hay defectos bloqueantes, críticos, mayores y los defectos menores y cosméticos son muy pocos, los que se ponen en la nota de Release.

Se confecciona el informe final (opcional).

Artefactos de testing

Estos artefactos son las salidas resultantes de los pasos anteriores, o sea del proceso de pruebas.

1. Plan de pruebas

Es el resultados de la planificación

Este plan, es análogo al plan de proyecto que se elabora en el enfoque tradicional (no forma parte de ese plan). En este plan, se definen:

- Datos necesarios para las pruebas.
- Recursos destinados a las pruebas.
- Herramientas a utilizar.
- Si se aplicará regresión (o no).
- Etc.

Para la confección de este plan no es necesario el código, sino que sólo se necesitan los requerimientos, en el formato que sea que se encuentren (ERS o Casos de Uso en métodos tradicionales, User Stories en Agile, etc.)

2. Casos de prueba:

Son el resultado del diseño

Es el artefacto fundamental del testing.

Consiste en una secuencia de pasos a seguir, las cuales describen un conjunto de acciones a realizar para lograr un resultado concreto y esperado.

Para esto se deben explicitar:

- a. las condiciones de inicio (las cuales fijan una situación particular),

- b. y los datos utilizados en ese escenario.

Se confeccionan con el objetivo de descubrir la mayor cantidad de defectos y minimizar el esfuerzo y tiempo para elaborarlos y ejecutarlos. Mientras no cambien los requerimientos, estos casos pueden ser utilizados las veces que sea necesario. La característica más importante de los casos de prueba es ser REPRODUCIBLES.

Si se encuentra un defecto y no es reproducible a través de un caso de prueba, no es un defecto.

Los casos de prueba se derivan de diferentes fuentes:

- c. Documentos del cliente;
- d. Información relevada;
- e. Requerimientos;
- f. Especificaciones de programación;
- g. Código.

3. Reporte de incidentes o defectos:

Es el resultado de la ejecución

Luego de la ejecución de los casos de prueba, se elabora un reporte, indicando cuáles fueron los casos de prueba que han pasado, y aquellos en los que se ha encontrado defectos, indicando cuáles fueron esos defectos encontrados y cómo reproducirlos para su detección y corrección. Este artefacto permite a los desarrolladores corregir esos defectos, para enviarlos nuevamente a testing.

4. Informe final:

Después del análisis de fallas

Suele contener métricas e información final del incremento del producto, se reporta cuántos ciclos y la cantidad de errores por ciclo, métodos, tipos de prueba utilizados, etc. Este es el único artefacto que es negociable en algunas organizaciones informales, se acuerda en la contratación del trabajo. Tiene utilidades estadísticas.

El Testing en el ciclo de vida del software

Es importante lograr involucrar las actividades del testing **lo más temprano posible en el ciclo de vida del software**. Si yo tengo definidos en principio los requisitos, ya puedo empezar a definir los casos de prueba.

Cuanto antes empezamos a realizar testing, podremos dar visibilidad más temprana al equipo sobre cómo se va a probar el producto y vamos a lograr disminuir costos de correcciones de defectos, ya que la idea sería encontrar errores, no defectos.

- Si desarrollamos con filosofía tradicional (Ciclo de Vida Secuencial) las pruebas se hacen al final ya que es el momento en el que nos entregan el producto.
- En cambio, si desarrollamos con ágil (Ciclo de Vida Iterativo/Incremental), hacemos testing por iteración. El problema es que se pueden incluir errores por la integración de los incrementos.

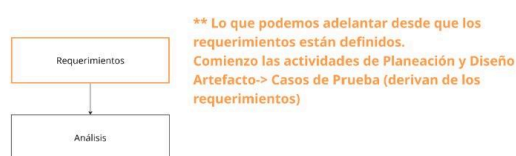
Recordar que **el objetivo de la prueba es VERIFICAR y VALIDAR**.

VALIDAR → ¿Estamos construyendo el sistema correcto? Sirve para ver si el sistema hace lo que tiene que hacer.

VERIFICAR → ¿Estamos construyendo el sistema correctamente? Sirve para ver que lo que el sistema haga, lo haga bien.

Por eso es importante armar los casos de prueba a partir de los **requerimientos**. Si no sabes que tiene que hacer el sistema no puedes validar.

Los errores de validación son más caros.



¿Cuánto testing es suficiente? ¿Cuándo dejar de probar?

Cuando dejar de probar se define en el plan de prueba.

Es IMPOSIBLE probar todas las opciones posibles, el testing exhaustivo no es viable. Decidir cuánto testing es suficiente depende de **una evaluación de nivel de riesgo y de costos asociados al proyecto**, depende del contexto del software y de su funcionalidad. Se define un criterio de aceptación, con colaboración del cliente, y en base a este criterio se determina cuánto testing se va a realizar. Puede ser un punto específico, un costo, un tiempo, etc.

Hay distintas estrategias para saber cuando dejar de probar:

- Tiempo de testeo disponible
- No tenga errores bloqueantes o graves y una cantidad aceptable de otros
- Cuando corrió el conjunto de casos de prueba que se definió por la versión.

El Criterio de Aceptación es lo que comúnmente se usa para resolver el problema de determinar cuándo una determinada fase de testing ha sido completada.

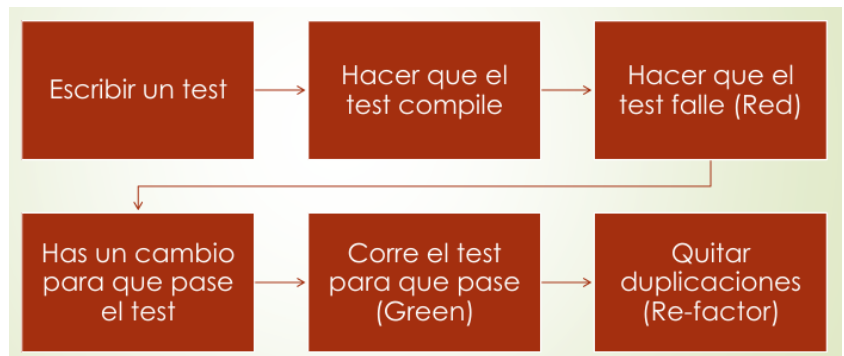
- Puede ser definido en términos de:
 - Costos
 - % de tests corridos sin fallas
 - Fallas predichas aún permanecen en el software
 - No hay defectos de una determinada severidad en el software

Principios del testing

- El testing no es una etapa que comienza al terminar de codificar. Lo ideal es que sea temprano en el ciclo de vida del proyecto.
- No es probar que el software funciona, sino encontrar defectos/errores.
- El tester no es el enemigo del programador, si bien tienen objetivos contrapuestos, el tester lo que hace es visibilizar lo que está sucediendo, muestra la presencia de defectos, no rompe el software.
- El testing ayuda a que el software sea confiable, no asegura la calidad.
- El testing exhaustivo es imposible.
- PARADOJA DEL PESTICIDA: Implica que cuando ejecutamos muchas veces los mismos casos de prueba, puede suceder que los casos de prueba se ejecuten sin fallas, pero que los verdaderos defectos no se verifiquen y queden ocultos.
- Es dependiente del contexto.
- Es una falacia decir que cuando se termina el testing hay ausencia de errores.
- Un programador debería evitar probar su propio código a excepción de las pruebas unitarias (único caso de excepción)
- Una unidad de programación no debería probar sus propios desarrollos

- No solo voy a buscar examinar lo que el software debería hacer sino también buscar examinar lo que pasa si el software no hace lo que debería hacer. (Ver qué pasa si hago un mal uso del software)
- No planificar el esfuerzo del testing sobre la suposición de que no se van a encontrar defectos, siempre ir con predisposición a encontrarlos.

TDD (Test Driven Development)



Es una práctica de desarrollo de software que se enfoca en escribir pruebas antes de desarrollar código. Se centra en el desarrollo funcional.

Desarrollo guiado por pruebas de software o Test-Driven Development es una técnica avanzada que involucra dos prácticas:

1. **Test First Development** (antes de escribir una línea de código, pienso en cómo voy a probar el correcto funcionamiento de ese código)
2. **Refactoring** (muchas veces voy escribiendo el código y lo desecho/ modifico para que cumpla con las pruebas, pero sin modificar su funcionalidad, eso es refactoring).

- Primero elijo el requerimiento que voy a probar
- Escribo la prueba
- Escribo la implementación de tal manera que logremos que el test compile
- Pero que este falle RED
- Luego hacemos el cambio necesario para que este pase GREEN
- Finalmente se refactoriza, implementando el código de la mejor manera posible.

Las 3 leyes del TDD:

1. **No escribir una línea de código hasta que no hayas escrito antes un test** case que falla
2. No es necesario escribir más test de los necesarios para que falle. Normalmente con un test que falle es suficiente.
3. No vas a escribir más código en Producción que el necesario para que el test pase



Mitos del testing

- El testing es una etapa que comienza al terminar de codificar.
- El testing es probar que el software funciona.
- TESTING = CALIDAD de PRODUCTO
- TESTING = CALIDAD de PROCESO
 - El testing NO asegura calidad ni le agrega calidad a algo que no la tiene.
- El tester es el enemigo del programador.
 - El testing NO asegura que no haya defectos, busca encontrar defectos

Testing en ambientes Ágiles (parte 2)

En los ambientes ágiles se adelanta la actividad de prueba, de manera que se avance con todo lo que se pueda referido a estas una vez que se cuenta con parte de los requerimientos, se trata de adelantar el pensamiento de las pruebas.

En el agilismo hay una tendencia a la automatización de las pruebas.

5 valores del Manifiesto de testing ágil

(Entender que la Prevención Durante la Construcción es Responsabilidad de todos)

Valoramos:

- **Pruebas DURANTE sobre pruebas al final:** En lugar de esperar a que el desarrollo termine para empezar a probar (como una fase separada al final), el testing se realiza continuamente durante todo el proceso de desarrollo. Integrar las pruebas desde el principio ayuda a encontrar problemas antes, cuando son más fáciles y baratos de arreglar. **No se ve como una fase aislada, sino como una actividad constante.** Contrasta con el modelo tradicional donde el testing tiene su propia columna o etapa final.
- **PREVENIR defectos sobre encontrar defectos:** En vez de centrarse sólo en descubrir errores una vez que ya están en el código, se busca participar activamente para evitar que esos errores se introduzcan desde el principio. Es más eficiente y menos costoso prevenir un error que corregirlo después.
- **ENTENDER lo que se está probando sobre verificar la funcionalidad:** No se trata sólo de marcar casillas y verificar si una función hace lo que dice la especificación (ser un "checker"). Es más valioso comprender el propósito, el contexto y el valor de lo que se está probando (ser un "tester"). **Entender el "para qué"** permite hacer pruebas más inteligentes y asegurar que el software realmente satisface las necesidades del usuario, no solo que cumple una lista de requisitos.
- **CONSTRUIR un mejor sistema sobre romper el sistema:** La mentalidad del tester ágil no es la de "romper el software". El objetivo es colaborar con todo el equipo para construir el mejor sistema posible desde una perspectiva de calidad.
- **Responsabilidad de TODO EL EQUIPO por la calidad sobre el tester es responsable de la calidad:** La calidad deja de ser responsabilidad exclusiva del tester o del equipo de QA. En un entorno ágil, *todo el equipo* (desarrolladores, testers, product owner, etc.) comparte la responsabilidad de entregar un producto de calidad. Evita que el tester se convierta en un cuello de botella o en el único "guardián de la calidad" al final del proceso. Promueve que todos piensen en la calidad desde el inicio y durante todo el desarrollo.

9 Principios para Testing en Proyectos Ágiles

FASE en la que TODOS se ADELANTAN a DOCUMENTAR y LIMPIAR para tener FEEDBACK y CONDUCIR hacia el DONE

1. **EL TESTING SE MUEVE HACIA ADELANTE EN EL PROYECTO:** Este principio contrasta directamente con el modelo tradicional (cascada), donde el testing es una de las últimas fases, ocurriendo justo antes del lanzamiento. **En agile, el testing se adelanta y se realiza de forma continua durante cada iteración. No se espera a que todo esté construido para empezar a probar.**
2. **TESTING NO ES UNA FASE:** El testing deja de ser una etapa secuencial y aislada para convertirse en una **actividad continua e integrada.**
3. **TODOS HACEN TESTING:** La calidad es una responsabilidad de todo el equipo. Aunque existe un rol de "Tester" con habilidades específicas no es la única persona que prueba.
4. **REDUCIR LA LATENCIA DEL FEEDBACK:** La "latencia del feedback" es el tiempo que transcurre entre que se introduce un error y se descubre. El testing ágil busca acortar este ciclo drásticamente a días o incluso horas.
5. **LAS PRUEBAS REPRESENTAN EXPECTATIVAS:** En lugar de ser solo una herramienta para encontrar fallos, las pruebas se convierten en la definición de lo que el software debe hacer.
6. **MANTENER EL CÓDIGO LIMPIO, CORREGIR LOS DEFECTOS RÁPIDO:** Prevenir errores antes de solucionarlos.
7. **REDUCIR LA SOBRECARGA DE DOCUMENTACIÓN EN LAS PRUEBAS:** El testing ágil prefiere la documentación ligera y funcional sobre los extensos planes de prueba manuales. Las propias pruebas automatizadas se convierten en la "documentación viva" del sistema
8. **LAS PRUEBAS SON PARTE DEL "DONE":** Una historia de usuario o una tarea no se considera "terminada" (Done) hasta que ha sido completamente probada y cumple con sus criterios de aceptación. El testing no es algo que ocurre después de que el desarrollo está "done"; es un requisito indispensable para alcanzar el estado de "done"
9. **DE PROBAR AL FINAL A "CONDUCIDO POR PRUEBAS":** Este principio resume el cambio fundamental. Se pasa de un modelo donde las pruebas validan el software al final, a un modelo donde las pruebas guían (conducen) el diseño y la implementación del software desde el inicio.

6 prácticas concretas para Testing en Proyectos Ágiles

Son la base para implementar los principios ágiles en las actividades de prueba:

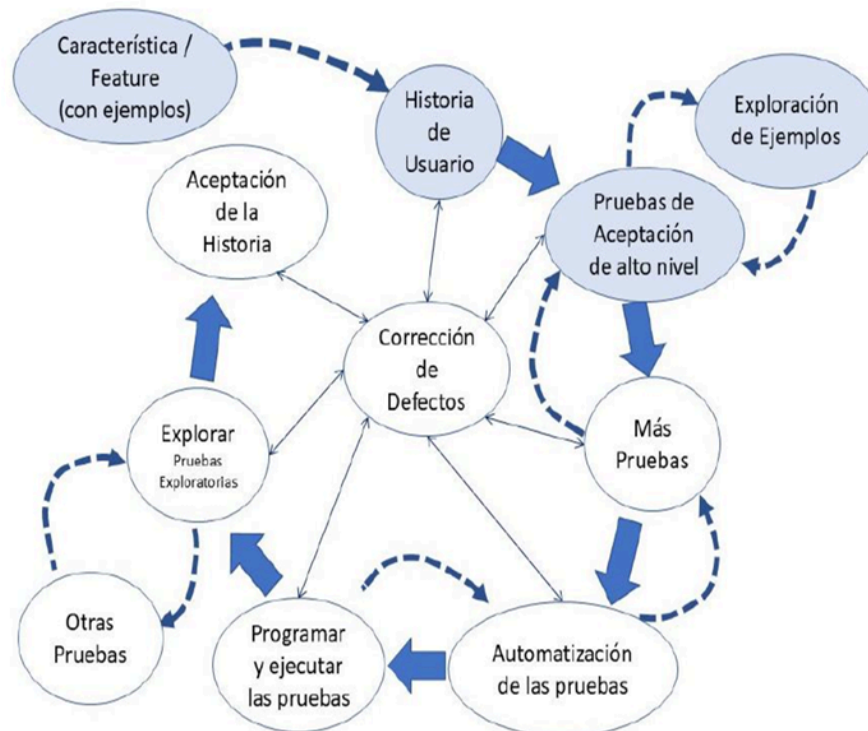
1. **TDD (Test Driven Development - Desarrollo Guiado por Pruebas):** Práctica de desarrollo en la que el programador escribe primero una prueba automatizada que falla (Rojo), luego escribe el código de producción mínimo necesario para que la prueba pase (Verde) y, finalmente, mejora el diseño del código sin cambiar su comportamiento (Refactorización)(Azul).

2. **ATDD (Acceptance Test Driven Development - Desarrollo Guiado por Pruebas de Aceptación):** Práctica colaborativa para definir los requisitos de una historia de usuario en forma de pruebas de aceptación antes de que comience el desarrollo.

Se preparan las pruebas antes de desarrollar una funcionalidad, y se ejecutan una vez que esa funcionalidad está implementada, es decir, cuando se tiene un incremento de producto.

Lo que busca es **validar que el software cumple los criterios de aceptación definidos en la User Story para confirmar que la funcionalidad implementada aporta el valor de negocio esperado.**

Esto lo hace tomando la feature, identificando la US y armando las pruebas de aceptación y explorando ejemplos hasta que tengo los necesarios, ahí hago más pruebas, automatizo, etc.



3. **Pruebas de Unidad e Integración Automatizadas:** Esta es la base de la pirámide del testing ágil. Los desarrolladores escriben una gran cantidad de pruebas de unidad automatizadas (que verifican pequeñas piezas de código, como un método) y pruebas de componente (que verifican la interacción entre varias piezas). Estas pruebas se ejecutan constantemente, a menudo con cada cambio en el código, a través de un proceso de Integración Continua.
4. **Pruebas de Regresión a Nivel de Sistema Automatizadas:** En un enfoque ágil, una selección de estas pruebas se automatiza para crear una "red de seguridad". Esta red de seguridad se ejecuta regularmente (por ejemplo, cada noche) para garantizar que las nuevas funcionalidades no hayan roto algo que ya existía. Esto permite al equipo hacer cambios y refactorizar el código con confianza.
5. **Pruebas Exploratorias:** Son una actividad donde el tester aprende, diseña y ejecuta pruebas simultáneamente, se hacen cuando se tienen dudas sobre el producto, o no se lo entiende del todo.
6. **Control de Versión de las Pruebas con el Código:** Esta práctica consiste en tratar el código de las pruebas automatizadas con la misma importancia que el código de

la aplicación. Las pruebas se guardan en el mismo sistema de control de versiones (como Git o Subversion) que el código fuente del producto. Esto garantiza que siempre se pueda asociar una versión específica del software con las pruebas que la validaron, lo cual es crucial para la reproducibilidad y el mantenimiento.

Modelo en V

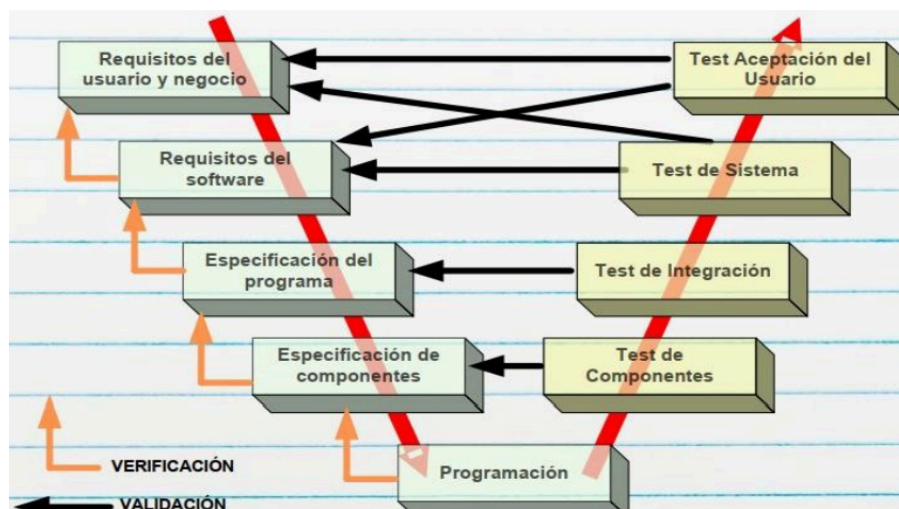
Este modelo se utiliza para poder determinar cuándo se está en condiciones de realizar determinadas actividades de testing, en función de la etapa de desarrollo en la que se encuentre el software. Acá se puede notar cómo las **pruebas se realizan en orden de granularidad inverso al proceso de desarrollo del software.**

Es decir, se desarrolla desde componentes de granularidad gruesa, como son los requerimientos abstraídos de detalles de implementación, hacia componentes de menor granularidad, como son clases o módulos, dependiendo del paradigma.

En cambio, para las pruebas, la granularidad se da en sentido inverso.

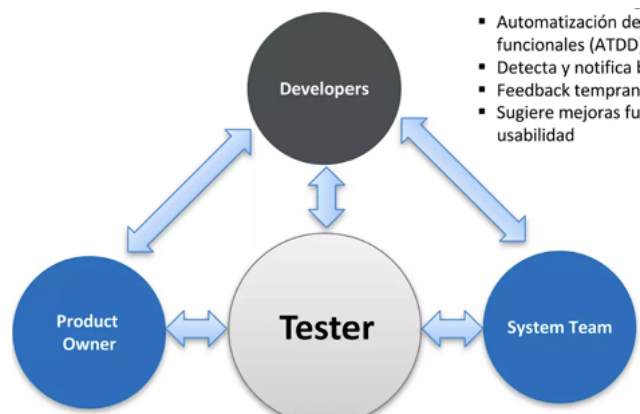
Testing de Componentes → Testing unitario.

Especificación del programa → diseño/arquitectura.



- **VERIFICACIÓN** La verificación apunta a ver si estamos construyendo el sistema correctamente. Libre de defectos, apuntando a la perfección de su funcionalidad. Ver si lo que hace lo hace bien.
- **VALIDACIÓN** La validación apunta a ver si estamos construyendo el sistema correcto, es decir, el sistema que cumple efectivamente con los requerimientos del cliente. Ver si hace lo que tiene que hacer.

El Rol del Tester en los Equipos Ágiles



El tester integra el equipo y colabora día a día con desarrolladores, analistas y product owners.

Sus responsabilidades principales incluyen:

- La preparación y mantenimiento del Ambiente de Pruebas
- La creación de Casos y Datos de Pruebas para las historias del Sprint
- La validación de "Done" de dichas historias.
- Registra y notificar los bugs
- Participa activamente en la Aceptación de la Historia junto al Product Owner (PO)
- Genera Métricas de Calidad.

El valor del tester se expande al ayudar a los otros roles:

- **Asiste al Product Owner** en tareas clave como:
 - La preparación y revisión de historias
 - La definición de los Criterios de Aceptación.

Esta colaboración permite un mejor entendimiento del Dominio de Negocio y ayuda a identificar dependencias tempranamente.

- En su **interacción con los Developers**, el tester ayuda a:
 - Definir estrategias de prueba
 - Participa en la automatización de pruebas funcionales (ATDD).

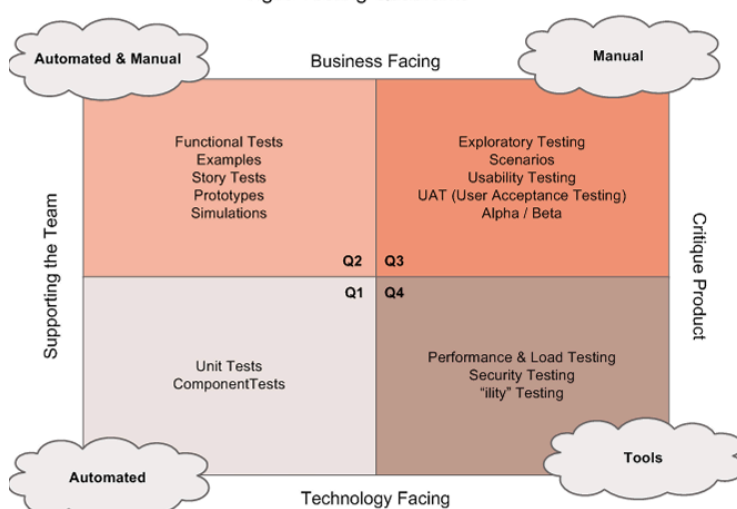
Su labor es crucial para proporcionar feedback temprano sobre las historias, detectar bugs y sugerir mejoras funcionales y de usabilidad.

Finalmente, **el tester se coordina con el System Team** para:

- Preparar la documentación de las Historias a Integrar y Probar
- Difundir el Plan de Pruebas de Integración.
- Ayuda a medir la completitud del Feature (característica)
- Gestiona la comunicación de los bugs que recibe, comunicándolos tanto al PO como a los Developers.

Cuadrantes de testing ágil

Agile Testing Quadrants



Pruebas orientadas al negocio

Pruebas que guían el desarrollo	Ejemplos Pruebas de Aceptación de la Historia Pruebas de Experiencia de Usuario (UX) Prototipos Simulaciones	Pruebas Exploratorias Flujos de Trabajo Pruebas de Usabilidad Pruebas de Aceptación (UAT) Monitoreo y Observabilidad
	Pruebas Unitarias Pruebas de Componentes (a nivel de código)	Pruebas de Rendimiento, Pruebas de Carga Pruebas de Seguridad Recuperabilidad Otros Atributos de Calidad (...ilidad)
Pruebas orientadas a la tecnología		

Pruebas que critican el producto

GUÍAN EL DESARROLLO

- Q1: pruebas que guían cómo se hace el código desde lo técnico, las escriben los desarrolladores
- Q2: también guían el desarrollo pero mirando el negocio, se asegura que estamos construyendo lo que hay que construir

CRITICAN EL PRODUCTO

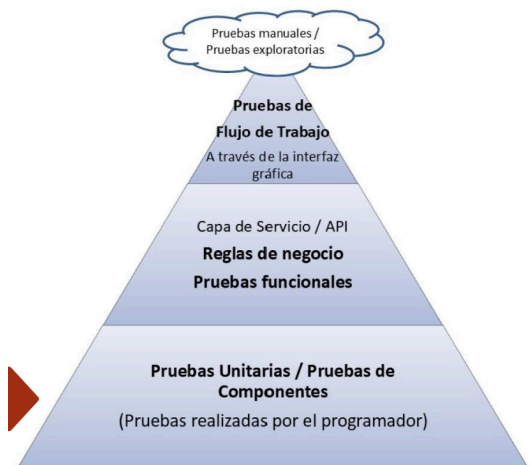
- Q3: Evalúan el producto como lo vería el usuario final
- Q4: Tecnología crítica, critican aspectos técnicos que no son la principal funcionalidad, suelen necesitar herramientas especiales

Pirámide de testing ágil

1. La Base: Pruebas Unitarias y de Componentes (Q1)

Esta es la capa más grande y fundamental de la pirámide.

- Qué son: Son pruebas escritas por los programadores que verifican las piezas más pequeñas de código (unidades o componentes) de forma aislada.
- Por qué en la base: Son muy rápidas de ejecutar (brindan feedback en segundos) y fáciles de mantener. La estrategia ágil dicta que la gran mayoría de las pruebas (alrededor del 80-90%) deberían estar en este nivel.
Red de seguridad principal para poder refactorizar y meter código nuevo con confianza.



2. El Medio: Pruebas de Aceptación / Servicio / API (Q1 y Q2)

Esta capa intermedia es más pequeña que la base.

- Qué son: Estas pruebas verifican las reglas de negocio, las APIs (capa de servicio) o la funcionalidad sin pasar por la interfaz de usuario. Incluyen pruebas de aceptación automatizadas (como en ATDD).
- Objetivo: Aseguran que diferentes componentes del sistema interactúan correctamente y que se cumplen los criterios de aceptación desde una perspectiva funcional.
- Por qué en el medio: Son más lentas que las pruebas unitarias porque involucran más partes del sistema, pero siguen siendo mucho más rápidas y estables que las pruebas de GUI.

3. La Cima: Pruebas de GUI y Manuales (Q2 y Q3)

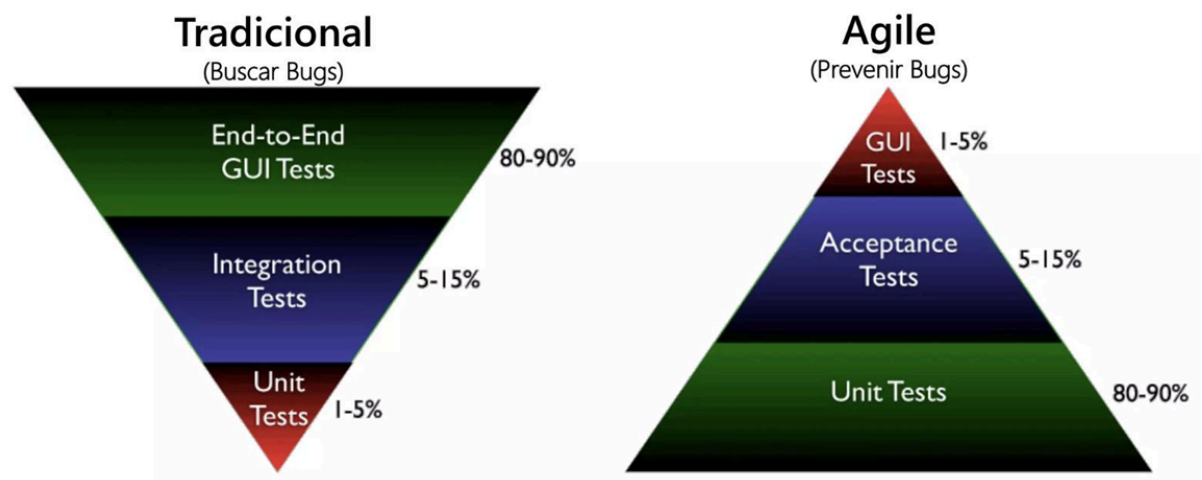
Esta es la capa más pequeña de la pirámide.

- Qué son: Simulan la interacción de una persona con el sistema a través de la pantalla. Incluyen pruebas automatizadas a través de la Interfaz Gráfica (GUI), pruebas de flujo de trabajo de extremo a extremo (End-to-End) y pruebas manuales, como las Pruebas Exploratorias.
- Objetivo: Las pruebas de GUI validan que la interfaz esté conectada correctamente y que los flujos de usuario funcionen. Las pruebas exploratorias se usan para "encontrar huecos" (find gaps) que las pruebas automatizadas no cubren.
- Por qué en la cima: Las pruebas de GUI son lentas, frágiles (se rompen fácilmente con cambios visuales) y costosas de mantener. Por eso, se busca tener muy pocas

(1-5%). Las pruebas manuales se reservan para tareas que no son prácticas de automatizar

Contraste con la Pirámide Tradicional

- Modelo Tradicional ("Cono de Helado"): Se invierte la pirámide.
 - La mayoría de las pruebas son manuales y a través de la GUI (80-90%),
 - con muy pocas pruebas unitarias (1-5%).Este enfoque es lento, costoso y encuentra los bugs tarde.
- Modelo Ágil ("Pirámide"):
 - La mayoría de las pruebas son unitarias (80-90%).Esto permite obtener feedback rápido, prevenir bugs desde el inicio y mantener un desarrollo sostenible.



Beneficios de la automatización del testing ágil

- **Rápida ejecución:** Las pruebas automatizadas se ejecutan mucho más rápido que las pruebas manuales. Tareas que a un equipo humano le tomarían días de regresión, un script las puede completar en minutos u horas, permitiendo obtener feedback de forma casi inmediata.
- **Reusabilidad:** Los scripts de prueba y los casos de prueba automatizados pueden ser reutilizados. Se pueden ejecutar en diferentes entornos (desarrollo, QA, producción), en diferentes versiones del software, o adaptarse para probar nuevas funcionalidades con modificaciones mínimas.
- **Mayor cobertura:** Al ser más rápidas, las pruebas automatizadas permiten ejecutar un conjunto mucho más amplio de casos de prueba en el mismo período de tiempo. Esto facilita la cobertura de más escenarios, combinaciones de datos y flujos de usuario que serían demasiado costosos o lentos de probar manualmente.
- **Mayor precisión:** La automatización elimina el error humano inherente a las tareas repetitivas. Un script ejecutará una prueba exactamente de la misma manera cada vez, sin omitir pasos ni cansarse. Esto asegura que los resultados sean consistentes y precisos.
- **Mayor alcance:** La automatización permite realizar tipos de pruebas que son impracticables para un humano. Por ejemplo, las pruebas de carga (simulando miles de usuarios concurrentes) o las pruebas de rendimiento complejas solo se pueden lograr eficazmente a través de la automatización, ampliando así el alcance del testing.

- **Reducción de costos:** Aunque requiere una inversión inicial, la automatización reduce los costos a largo plazo. Al encontrar defectos más temprano (cuando son más baratos de arreglar), reducir las horas de trabajo manual y acelerar los ciclos de lanzamiento, el retorno de la inversión es significativo.

La automatización se vuelve indispensable, le permiten a las personas enfocarse en otro tipo de pruebas y hacer regresión.

Revisiones Técnicas

Las revisiones técnicas son una actividad de verificación **estática del software** que consiste en examinar cualquier producto de trabajo (requerimientos, diseño, código, casos de prueba, documentación, etc.) con el fin de identificar defectos, omisiones, ambigüedades o inconsistencias. Son un proceso de V&V estático, lo que significa que se aplican a cualquier representación legible del software (como documentos de requerimientos, diseño o código) sin necesidad de ejecutar el programa

Principales características:

- Son parte del proceso de verificación, mientras que las pruebas corresponden a la validación.
- Se aplican a cualquier representación legible del software, incluso versiones incompletas
- **Previenen fallas** en lugar de detectarlas tarde; cuanto antes se encuentre un defecto, menor es el costo de corregirlo.

Tipos de revisiones

- **Formales:** tienen un proceso definido con roles.
 - Inspecciones (Inspección de código de Fagan e inspección de Gilb).
- **Informales:** cuando no existe un proceso de cómo realizarlo.
 - Walkthrough o recorrido.

Walkthrough o recorrido (informal)

Un diseñador o programador dirige miembros del equipo de desarrollo y otras partes interesadas a través de un producto de software, donde **los participantes formulan preguntas y realizan comentarios acerca de posibles errores** y otros problemas. No existe un proceso formal. Consiste en reuniones informales de colegas donde se debaten las correcciones a aplicar al producto de trabajo. No hay control del proceso.

Tiene los siguientes objetivos:

1. Mínima Sobrecarga
2. Capacitación de Desarrolladores
3. Rápido retorno

Esta técnica **no obtiene métricas para aprender y dejar registros**. Sin embargo, es una de las técnicas más elegidas en los enfoques ágiles. Hay una junta de revisión entre colegas después de completar cada iteración del software (una revisión rápida), en la que pueden exponerse los conflictos y problemas de calidad sobre el producto.

Inspecciones (formal)

Tiene un **proceso formal** y cuenta con un **conjunto de roles**. Es necesario la utilización de un checklist, que ayuda a la memoria para saber qué cosas controlar. Se toman métricas y

finalmente **se realiza un reporte de la revisión al final de la inspección para analizar los defectos encontrados.**

Es una actividad que garantiza la calidad del software, cuyo éxito depende de la planificación.

Tiene como objetivos:

1. Descubrir errores.
2. Verificar que el software alcanza sus requerimientos.
3. Garantizar que el software fue construido de acuerdo con ciertos estándares.
4. Conseguir un software desarrollado de manera uniforme.
5. Hacer que los proyectos sean más manejables.

Son procesos time boxing y exigen un alto esfuerzo intelectual.

Etapas del proceso de inspección

1. Planificación: el moderador, **a pedido del autor**, planifica la inspección definiendo el lugar, el tiempo de duración y los roles. La duración de las reuniones no debe superar las 2 horas, dado que la inspección es un proceso de alto esfuerzo intelectual.

2. Visión general: esta etapa es opcional. El autor realiza una descripción general del producto a inspeccionar.

3. Preparación: es la preparación de cada rol para la reunión. Cada rol adquiere una copia del producto de trabajo que deberá leer y analizar, con el fin de encontrar potenciales defectos.

4. Reunión de inspección:

1. El equipo realiza un análisis para recolectar los potenciales defectos previos y descartar falsos positivos.
2. El lector lee el producto de trabajo y los inspectores comparten los defectos encontrados, los cuales son registrados por el anotador.
3. La **reunión finaliza con una conclusión acerca de si se acepta o no el producto de trabajo** inspeccionado.
4. Finalmente se realiza un informe detallando que se revisó, por quién, que se descubrió y que se concluyó.

5. Corrección: finalizada la reunión, el autor realiza las correcciones de los defectos encontrados .

6. Seguimiento: Dependiendo de la gravedad puede existir un proceso de re-inspección. De lo contrario, si el defecto era muy simple, el autor simplemente lo corrige.

Roles participantes

Al ser una técnica formal, si o si debe contar con los siguientes roles:

1. **Autor:** creador o encargado de mantener el producto a inspeccionar. Inicia el proceso seleccionando a un moderador y junto a este eligen al resto de los roles. Entrega el producto a ser inspeccionado al moderador.
2. **Moderador:** planifica y lidera la revisión. Trabaja junto al autor para elegir los demás roles. Entrega el producto a inspeccionar al inspector 2 días antes de la reunión. Coordina la reunión de forma que no ocurran conductas inapropiadas y realiza un seguimiento de los defectos encontrados.
3. **Anotador:** registra los hallazgos de la inspección. Usualmente termina confeccionado el reporte de la revisión.
4. **Lector:** lee el producto a ser inspeccionado. Este rol es necesario para que los participantes no se dispersen.

5. **Inspector:** Examina el producto antes de la reunión para encontrar defectos. Registra sus tiempos de preparación. todos pueden ser inspectores. Todos pueden inspeccionar.

Ciertos roles pueden ser asumidos por la misma persona.

Unidad 2: Gestión Lean Agile de Productos de Software

- Framework para escalar SCRUM / Nexus
- Filosofía Lean - Método Kanban en el contexto del desarrollo de software
- Métricas en los 3 enfoques (Tradicional / Lean / Agile)

Unidad 3: Gestión de software como producto

- Prácticas continuas

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

- Calidad de Producto:
 - Planificación de pruebas para el software- Niveles y tipos de pruebas para el software.
 - Técnicas y herramientas para probar software.
 - Testing agile
 - Técnicas y Herramientas para la realización de revisiones técnicas del software.